Mini project report on

Insurance Claims Processing System

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS351A – DBMS Project

Submitted by:

Shreya Shenoy	**PES2UG24CS487**
Shreya Patil	**PES2UG24CS484**

under the guidance of

Dr. Shilpa S

Assistant Professor

PES University

AUG - DEC 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Insurance Claims Processing System

is a bonafide work carried out by

Shreya Shenoy	**PES2UG24CS487**
Shreya Patil	**PES2UG24CS484**

In partial fulfilment for the completion of fifth semester DBMS Project (UE24CS351A) in the Program of Study -Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period AUG. 2026 – DEC. 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 5th semester academic requirements in respect of project work.

Signature
Dr. Shilpa S
Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Insurance Claims Processing System** has been carried out by us under the guidance of **Dr. Shilpa S, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester AUG – DEC 2026.

Shreya Shenoy	**PES2UG24CS487**	**<Shreya Shenoy>**
Shreya Patil	**PES2UG24CS484**	**<Shreya Patil>**

ACKNOWLEDGEMENT

I would like to express my gratitude to Dr. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS351A - DBMS Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

The Insurance Claims Processing System is a database-driven application designed to organize and manage the various activities involved in insurance claim processing. The system addresses the need for efficient management of customer information, insurance policies, insured assets, incidents, claims, assessments, settlements and payments.

The project uses a relational database implemented using MySQL to maintain structured insurance records and establish relationships between different entities through primary keys, foreign keys and integrity constraints. The database consists of ten interconnected tables: CUSTOMER, POLICY_TYPE, INSURANCE_POLICY, INSURED_ASSET, INCIDENT, CLAIM, ASSESSOR, CLAIM_ASSESSMENT, CLAIM_SETTLEMENT and PAYMENT.

The system demonstrates important database concepts including Data Definition Language (DDL), Data Manipulation Language (DML), joins, aggregate functions, grouping, subqueries, stored procedures, stored functions, triggers, views, indexes and transaction management.

A web-based application is being developed using Next.js, React and TypeScript, with API routes facilitating communication between the application and the MySQL database. The interface provides an administrative dashboard and dedicated modules for accessing customer, policy and claim information.

The project aims to demonstrate how appropriate database design, relational operations and application integration can support the systematic management of insurance claim information while maintaining data consistency, integrity and accessibility.

Keywords: Insurance Claims, MySQL, Database Management System, Relational Database, Next.js, SQL, Database Integration.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1.	INTRODUCTION	9-10
2.	PROBLEM DEFINITION	11-12
3.	ER MODEL	13-14
4.	ER TO RELATIONAL MAPPING	15-18
5.	DDL STATEMENTS	19-27
6.	DML STATEMENTS	28-32
7.	QUERIES	33-35
8.	STORED PROCEDURE, FUNCTIONS AND TRIGGERS	36-38
9.	MISCELLANEOUS QUERIES	39-52
10.	FRONT END DEVELOPMENT	53-55
REFERENCES/BIBLIOGRAPHY		56
APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS		57-58

LIST OF TABLES

Table No.	Title	Page No.
10.A	Appendix A	55-57

LIST OF FIGURES

Figure No.	Title	Page No.
3.1	Entity Relationship Diagram	**13**
4.1	Complete Relational Schema	**17**
NA	All query screenshots and frontend screenshots	**19-54**

1. INTRODUCTION

1.1 Overview

The Insurance Claims Processing System manages information related to customers, insurance policies, incidents, claims, assessments, settlements and payments through a centralized relational database.

It uses MySQL for data storage and a Next.js-based web application for accessing and managing records.

1.2 Motivation

Insurance claim processing involves multiple interconnected records. Managing these separately can lead to data duplication, inconsistencies and difficulties in retrieving information.

This project aims to organize these records using a structured database system.

1.3 Objectives

- Design and implement a relational database for insurance claims.
- Establish relationships using primary and foreign keys.
- Maintain customer, policy, claim and payment records.
- Implement SQL queries and advanced database features.
- Develop a web interface integrated with MySQL.
- Demonstrate data integrity and efficient information retrieval.

1.4 Scope

The project covers:

- Customer and policy management.
- Insured asset and incident records.
- Claim registration and assessment.
- Settlement and payment records.
- SQL-based retrieval and analysis.
- Web-based access to database information.

1.5 Database Technology

MySQL is used as the primary database due to its support for relational data, constraints, joins, transactions and advanced SQL features.

A NoSQL database component: MongoDB is also included as part of the Level 3 project requirements.

2. PROBLEM DEFINITION

2.1 Problem Statement

Insurance claim processing involves managing interconnected information about customers, policies, incidents, claims, assessments and payments. Maintaining these records separately makes retrieval, tracking and consistency difficult.

The project aims to develop a centralized database system that organizes insurance records and supports efficient claim information management.

2.2 Existing System

Fragmented record management can result in:

- Data duplication and inconsistency.
- Difficulty retrieving related records.
- Manual claim tracking.
- Delays in accessing settlement and payment information.

2.3 Proposed System

The proposed system uses a relational database with interconnected tables to manage insurance records. SQL queries enable retrieval and analysis of related information, while a web-based interface provides access to the system.

2.4 Functional Requirements

1. Maintain customer and policy records.
2. Record insured assets and incidents.
3. Maintain insurance claim details.
4. Record claim assessments.
5. Manage settlements and payments.
6. Retrieve related information using SQL queries.
7. Generate summaries using aggregate operations.
8. Provide a web-based interface for accessing records.

2.5 Non-Functional Requirements

- Integrity: Maintain valid relationships between records.
- Security: Protect database credentials and access.
- Reliability: Ensure consistent database operations.
- Usability: Provide a simple interface.
- Performance: Enable efficient data retrieval.
- Maintainability: Keep the application modular.

3. ER MODEL

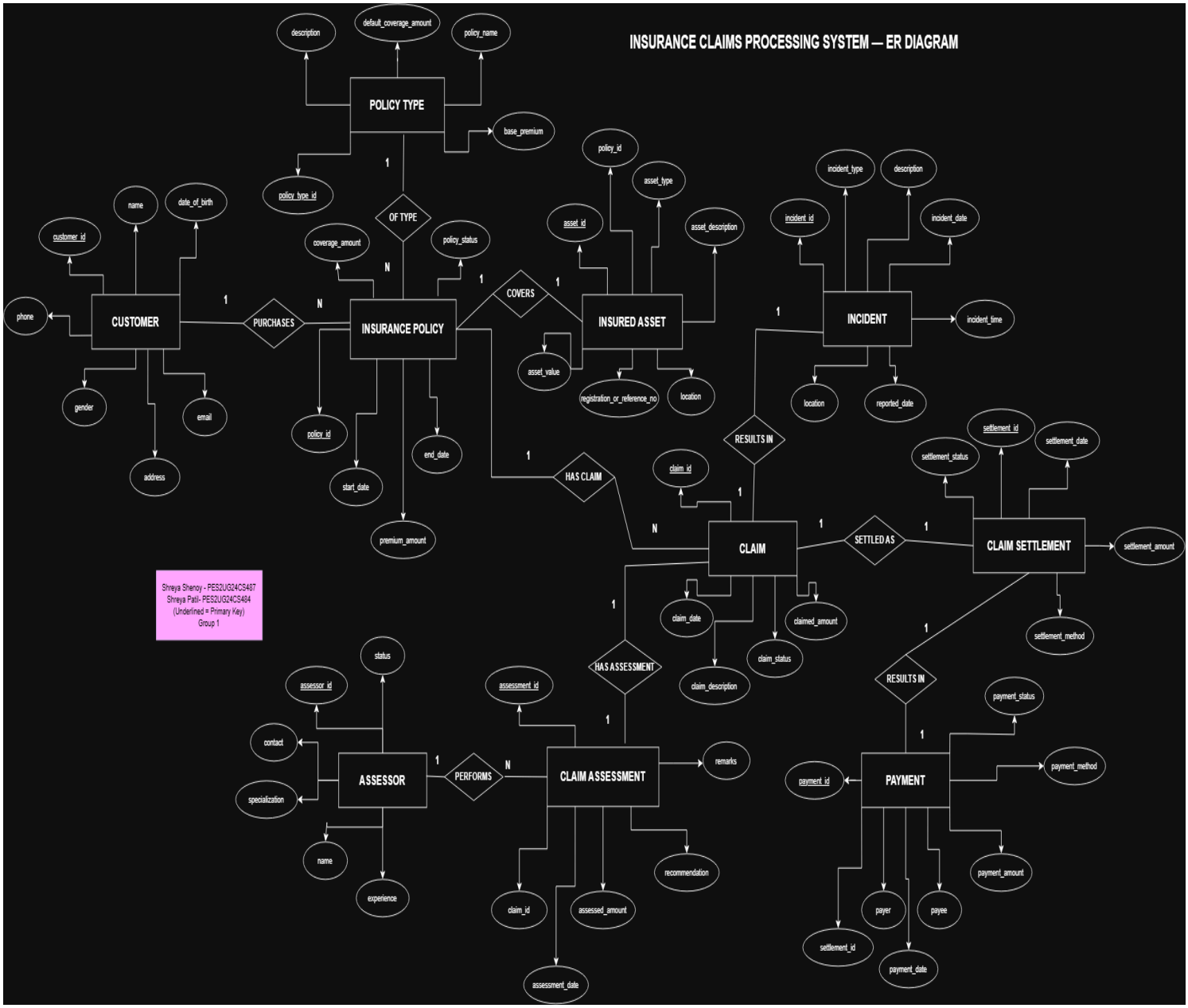


Figure 3.1: Entity Relationship Diagram

The system consists of ten entities representing the major components of insurance claim processing.

The entities are:

1. CUSTOMER

2. POLICY_TYPE
3. INSURANCE_POLICY
4. INSURED_ASSET
5. INCIDENT
6. CLAIM
7. ASSESSOR
8. CLAIM_ASSESSMENT
9. CLAIM_SETTLEMENT
10. PAYMENT

3.2 Relationships

The major relationships between entities are:

- A customer can hold multiple insurance policies.
- Each insurance policy belongs to a policy type.
- A policy can cover multiple insured assets.
- A policy can be associated with multiple claims.
- An incident can be associated with claims.
- Claims can have assessments performed by assessors.
- Claims can have associated settlements.
- Settlements can have associated payments.

4. ER TO RELATIONAL MAPPING

4.1 Steps of algorithm for chosen problem

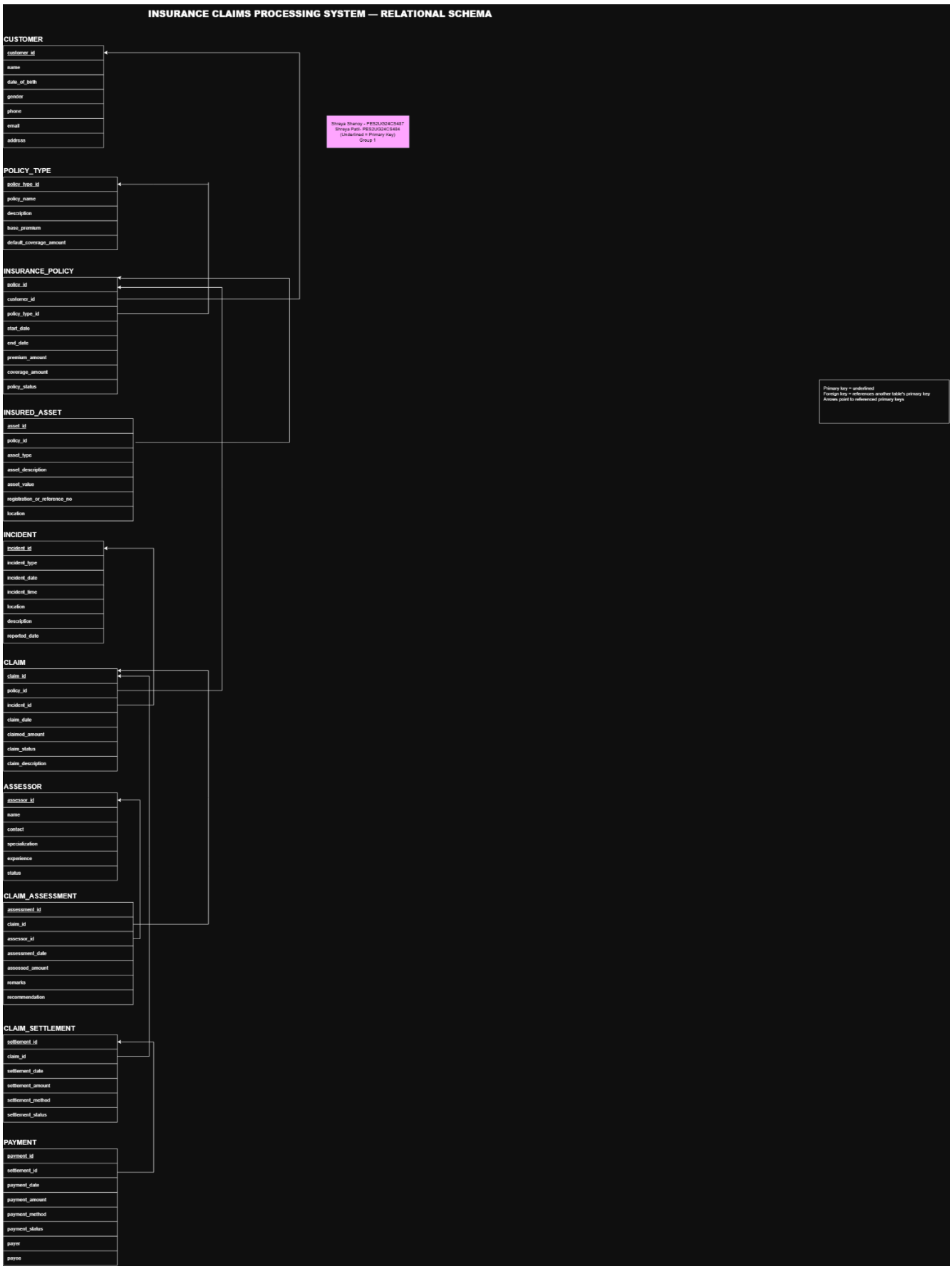
1. Start the system and establish a connection with the MySQL database.
2. Retrieve and maintain customer details.
3. Store insurance policy details and associate each policy with its customer and policy type.
4. Record insured assets under their respective policies.
5. Record incident details when an insured event occurs.
6. Register a claim by associating it with the relevant policy and incident.
7. Assign an assessor to evaluate the claim.
8. Record the assessment details and assessed amount.
9. Determine and record the settlement amount for the claim.
10. Record payments associated with the settlement.
11. Retrieve and display claim, assessment, settlement and payment information using SQL queries.
12. Generate relevant summaries and reports from the stored records.
13. End the process.

4.2 Relational Schema

The resulting database consists of the following ten relations:

- CUSTOMER
- POLICY_TYPE
- INSURANCE_POLICY
- INSURED_ASSET
- INCIDENT
- CLAIM
- ASSESSOR

- CLAIM_ASSESSMENT
- CLAIM_SETTLEMENT


INSURANCE CLAIMS PROCESSING SYSTEM — RELATIONAL SCHEMA
CUSTOMER
customer_id
name
date_of_birth
gender
phone
email
address
POLICY_TYPE
policy_type_id
policy_name
description
base_premium
default_coverage_amount
INSURANCE_POLICY
policy_id
customer_id
policy_type_id
start_date
end_date
premium_amount
coverage_amount
policy_status
INSURED_ASSET
asset_id
policy_id
asset_type
asset_description
asset_value
registration_or_reference_no
location
INCIDENT
incident_id
incident_type
incident_date
incident_time
location
description
reported_date
CLAIM
claim_id
policy_id
incident_id
claim_date
claimed_amount
claim_status
claim_description
ASSESSOR
assessor_id
name
contact
specialization
experience
status
CLAIM_ASSESSMENT
assessment_id
claim_id
assessor_id
assessment_date
assessed_amount
remarks
recommendation
CLAIM_SETTLEMENT
settlement_id
claim_id
settlement_date
settlement_amount
settlement_method
settlement_status
PAYMENT
payment_id
settlement_id
payment_date
payment_amount
payment_method
payment_status
payer
payee
Shreya Shenoy - PES2UG24CS487
Shreya Patil- PES2UG24CS484
(Underlined = Primary Key)
Group 1
Primary key = underlined
Foreign key = references another table's primary key
Arrows point to referenced primary keys

Insert Figure 4.1: Complete Relational Schema

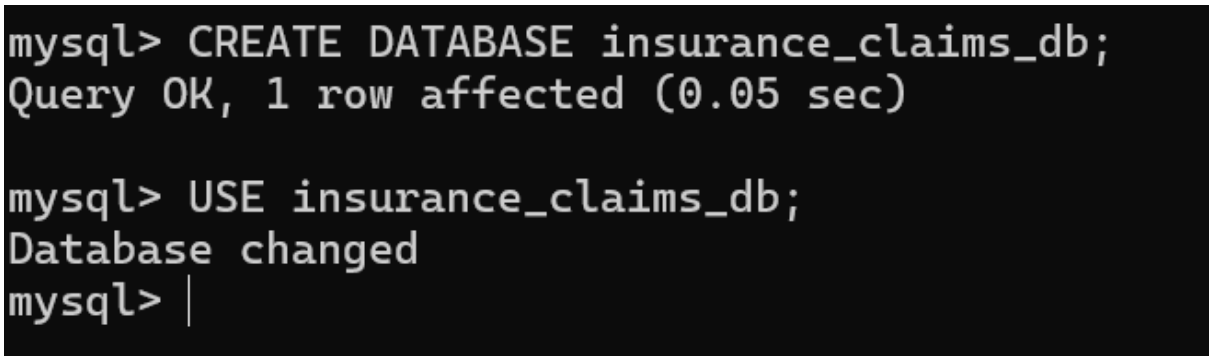
5. DDL STATEMENTS

Data Definition Language (DDL) statements are used to create and modify database structures.

The project uses the following operations:

- CREATE DATABASE – Creates the database.
- CREATE TABLE – Defines the ten relational tables.
- PRIMARY KEY – Uniquely identifies each record.
- FOREIGN KEY – Establishes relationships between tables.
- Constraints – Maintain data validity.
- ALTER TABLE – Adds the claim_priority attribute to CLAIM.
- CREATE VIEW – Creates a consolidated claim summary.
- CREATE INDEX – Improves retrieval based on claim status and date.
- DROP TABLE – Removes the temporary staging table.

1)



database created and being used

2) rechecking if the database we are using currently is the one we just made

```
mysql> SELECT DATABASE();
+----------------------+
| DATABASE()           |
+----------------------+
| insurance_claims_db  |
+----------------------+
1 row in set (0.00 sec)

mysql>
```

3) creating and populating tables for those entities which have no dependencies on other tables through foreign keys

i) CUSTOMER table created and populated

```
mysql> CREATE TABLE CUSTOMER (
    ->     customer_id INT PRIMARY KEY,
    ->     name VARCHAR(100) NOT NULL,
    ->     date_of_birth DATE,
    ->     gender VARCHAR(10),
    ->     phone VARCHAR(15),
    ->     email VARCHAR(100),
    ->     address VARCHAR(255)
    -> );
Query OK, 0 rows affected (0.17 sec)

mysql> DESC CUSTOMER;
+---------------+--------------+------+-----+---------+-------+
| Field         | Type         | Null | Key | Default | Extra |
+---------------+--------------+------+-----+---------+-------+
| customer_id   | int          | NO   | PRI | NULL    |       |
| name          | varchar(100) | NO   |     | NULL    |       |
| date_of_birth | date         | YES  |     | NULL    |       |
| gender        | varchar(10)  | YES  |     | NULL    |       |
| phone         | varchar(15)  | YES  |     | NULL    |       |
| email         | varchar(100) | YES  |     | NULL    |       |
| address       | varchar(255) | YES  |     | NULL    |       |
+---------------+--------------+------+-----+---------+-------+
7 rows in set (0.08 sec)
```

ii) POLICY_TYPE table created and populated

```
mysql> CREATE TABLE POLICY_TYPE (
    ->     policy_type_id INT PRIMARY KEY,
    ->     policy_name VARCHAR(100) NOT NULL,
    ->     description VARCHAR(255),
    ->     base_premium DECIMAL(12,2),
    ->     default_coverage_amount DECIMAL(15,2)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql> DESC POLICY_TYPE;
+------------------------+---------------+------+-----+---------+-------+
| Field                  | Type          | Null | Key | Default | Extra |
+------------------------+---------------+------+-----+---------+-------+
| policy_type_id         | int           | NO   | PRI | NULL    |       |
| policy_name            | varchar(100)  | NO   |     | NULL    |       |
| description            | varchar(255)  | YES  |     | NULL    |       |
| base_premium           | decimal(12,2) | YES  |     | NULL    |       |
| default_coverage_amount | decimal(15,2) | YES  |     | NULL    |       |
+------------------------+---------------+------+-----+---------+-------+
5 rows in set (0.00 sec)

mysql>
```

iii)INCIDENT table created and populated

```
mysql> CREATE TABLE INCIDENT (
    ->     incident_id INT PRIMARY KEY,
    ->     incident_type VARCHAR(100) NOT NULL,
    ->     incident_date DATE NOT NULL,
    ->     incident_time TIME,
    ->     location VARCHAR(150),
    ->     description VARCHAR(500),
    ->     reported_date DATE
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql> DESC INCIDENT;
+---------------+--------------+------+-----+---------+-------+
| Field         | Type         | Null | Key | Default | Extra |
+---------------+--------------+------+-----+---------+-------+
| incident_id   | int          | NO   | PRI | NULL    |       |
| incident_type | varchar(100) | NO   |     | NULL    |       |
| incident_date | date         | NO   |     | NULL    |       |
| incident_time | time         | YES  |     | NULL    |       |
| location      | varchar(150) | YES  |     | NULL    |       |
| description   | varchar(500) | YES  |     | NULL    |       |
| reported_date | date         | YES  |     | NULL    |       |
+---------------+--------------+------+-----+---------+-------+
7 rows in set (0.00 sec)

mysql>
```

iv)ASSESSOR table created and populated

```
mysql> CREATE TABLE ASSESSOR (
    ->     assessor_id INT PRIMARY KEY,
    ->     name VARCHAR(100) NOT NULL,
    ->     contact VARCHAR(15),
    ->     specialization VARCHAR(100),
    ->     experience INT,
    ->     status VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql> DESC ASSESSOR;
+----------------+--------------+------+-----+---------+-------+
| Field          | Type         | Null | Key | Default | Extra |
+----------------+--------------+------+-----+---------+-------+
| assessor_id    | int          | NO   | PRI | NULL    |       |
| name           | varchar(100) | NO   |     | NULL    |       |
| contact        | varchar(15)  | YES  |     | NULL    |       |
| specialization | varchar(100) | YES  |     | NULL    |       |
| experience     | int          | YES  |     | NULL    |       |
| status         | varchar(30)  | YES  |     | NULL    |       |
+----------------+--------------+------+-----+---------+-------+
6 rows in set (0.01 sec)

mysql>
```

confirming all the tables we have made so far

```
mysql> SHOW TABLES;
+-----------------------------------+
| Tables_in_insurance_claims_db     |
+-----------------------------------+
| assessor                          |
| customer                          |
| incident                          |
| policy_type                       |
+-----------------------------------+
4 rows in set (0.03 sec)

mysql>
```

4) creating and making entries for all the dependent tables with foreign keys

i)INSURANCE_POLICY table created and populated

```
mysql> CREATE TABLE INSURANCE_POLICY (
    ->     policy_id INT PRIMARY KEY,
    ->     customer_id INT NOT NULL,
    ->     policy_type_id INT NOT NULL,
    ->     start_date DATE NOT NULL,
    ->     end_date DATE NOT NULL,
    ->     premium_amount DECIMAL(12,2) NOT NULL,
    ->     coverage_amount DECIMAL(15,2) NOT NULL,
    ->     policy_status VARCHAR(30) NOT NULL,
    ->
    ->     FOREIGN KEY (customer_id)
    ->         REFERENCES CUSTOMER(customer_id),
    ->
    ->     FOREIGN KEY (policy_type_id)
    ->         REFERENCES POLICY_TYPE(policy_type_id)
    -> );
Query OK, 0 rows affected (0.17 sec)

mysql> DESC INSURANCE_POLICY;
+-----------------+---------------+------+-----+---------+-------+
| Field           | Type          | Null | Key | Default | Extra |
+-----------------+---------------+------+-----+---------+-------+
| policy_id       | int           | NO   | PRI | NULL    |       |
| customer_id     | int           | NO   | MUL | NULL    |       |
| policy_type_id  | int           | NO   | MUL | NULL    |       |
| start_date      | date          | NO   |     | NULL    |       |
| end_date        | date          | NO   |     | NULL    |       |
| premium_amount  | decimal(12,2) | NO   |     | NULL    |       |
| coverage_amount | decimal(15,2) | NO   |     | NULL    |       |
| policy_status   | varchar(30)   | NO   |     | NULL    |       |
+-----------------+---------------+------+-----+---------+-------+
8 rows in set (0.00 sec)

mysql>
```

ii)INSURED_ASSET table is created and populated

```
mysql> CREATE TABLE INSURED_ASSET (
    ->     asset_id INT PRIMARY KEY,
    ->     policy_id INT NOT NULL,
    ->     asset_type VARCHAR(50) NOT NULL,
    ->     asset_description VARCHAR(255),
    ->     asset_value DECIMAL(15,2),
    ->     registration_or_reference_no VARCHAR(100),
    ->     location VARCHAR(150),
    ->
    ->     FOREIGN KEY (policy_id)
    ->         REFERENCES INSURANCE_POLICY(policy_id),
    ->
    ->     UNIQUE (policy_id)
    -> );
Query OK, 0 rows affected (0.15 sec)

mysql> DESC INSURED_ASSET;
+-----------------------------+---------------+------+-----+---------+-------+
| Field                       | Type          | Null | Key | Default | Extra |
+-----------------------------+---------------+------+-----+---------+-------+
| asset_id                    | int           | NO   | PRI | NULL    |       |
| policy_id                   | int           | NO   | UNI | NULL    |       |
| asset_type                  | varchar(50)   | NO   |     | NULL    |       |
| asset_description           | varchar(255)  | YES  |     | NULL    |       |
| asset_value                 | decimal(15,2) | YES  |     | NULL    |       |
| registration_or_reference_no | varchar(100)  | YES  |     | NULL    |       |
| location                    | varchar(150)  | YES  |     | NULL    |       |
+-----------------------------+---------------+------+-----+---------+-------+
7 rows in set (0.01 sec)

mysql>
```

unique keyword is being used here since the relationship is 1:1 and not 1:n

iii) CLAIM table is created and populated

```
mysql> CREATE TABLE CLAIM (
    ->     claim_id INT PRIMARY KEY,
    ->     policy_id INT NOT NULL,
    ->     incident_id INT NOT NULL,
    ->     claim_date DATE NOT NULL,
    ->     claimed_amount DECIMAL(15,2) NOT NULL,
    ->     claim_status VARCHAR(30) NOT NULL,
    ->     claim_description VARCHAR(500),
    ->
    ->     FOREIGN KEY (policy_id)
    ->         REFERENCES INSURANCE_POLICY(policy_id),
    ->
    ->     FOREIGN KEY (incident_id)
    ->         REFERENCES INCIDENT(incident_id),
    ->
    ->     UNIQUE (incident_id)
    -> );
Query OK, 0 rows affected (0.14 sec)

mysql> DESC CLAIM;
+-------------------+---------------+------+-----+---------+-------+
| Field             | Type          | Null | Key | Default | Extra |
+-------------------+---------------+------+-----+---------+-------+
| claim_id          | int           | NO   | PRI | NULL    |       |
| policy_id         | int           | NO   | MUL | NULL    |       |
| incident_id       | int           | NO   | UNI | NULL    |       |
| claim_date        | date          | NO   |     | NULL    |       |
| claimed_amount    | decimal(15,2) | NO   |     | NULL    |       |
| claim_status      | varchar(30)   | NO   |     | NULL    |       |
| claim_description | varchar(500)  | YES  |     | NULL    |       |
+-------------------+---------------+------+-----+---------+-------+
7 rows in set (0.01 sec)
```

iv) CLAIM_ASSESSMENT is created and populated

```
mysql> CREATE TABLE CLAIM_ASSESSMENT (
    ->     assessment_id INT PRIMARY KEY,
    ->     claim_id INT NOT NULL,
    ->     assessor_id INT NOT NULL,
    ->     assessment_date DATE NOT NULL,
    ->     assessed_amount DECIMAL(15,2) NOT NULL,
    ->     remarks VARCHAR(500),
    ->     recommendation VARCHAR(100),
    ->
    ->     FOREIGN KEY (claim_id)
    ->         REFERENCES CLAIM(claim_id),
    ->
    ->     FOREIGN KEY (assessor_id)
    ->         REFERENCES ASSESSOR(assessor_id),
    ->
    ->     UNIQUE (claim_id)
    -> );
Query OK, 0 rows affected (0.13 sec)

mysql> DESC CLAIM_ASSESSMENT;
+-----------------+---------------+------+-----+---------+-------+
| Field           | Type          | Null | Key | Default | Extra |
+-----------------+---------------+------+-----+---------+-------+
| assessment_id   | int           | NO   | PRI | NULL    |       |
| claim_id        | int           | NO   | UNI | NULL    |       |
| assessor_id     | int           | NO   | MUL | NULL    |       |
| assessment_date | date          | NO   |     | NULL    |       |
| assessed_amount | decimal(15,2) | NO   |     | NULL    |       |
| remarks         | varchar(500)  | YES  |     | NULL    |       |
| recommendation  | varchar(100)  | YES  |     | NULL    |       |
+-----------------+---------------+------+-----+---------+-------+
7 rows in set (0.01 sec)
```

once again UNIQUE is being used because of the 1:1 relationship

v)CLAIM_SETTLEMENT table is created and populated with attributes

```
mysql> CREATE TABLE CLAIM_SETTLEMENT (
    ->     settlement_id INT PRIMARY KEY,
    ->     claim_id INT NOT NULL,
    ->     settlement_date DATE,
    ->     settlement_amount DECIMAL(15,2) NOT NULL,
    ->     settlement_method VARCHAR(50),
    ->     settlement_status VARCHAR(30),
    ->
    ->     FOREIGN KEY (claim_id)
    ->         REFERENCES CLAIM(claim_id),
    ->
    ->     UNIQUE (claim_id)
    -> );
Query OK, 0 rows affected (0.11 sec)

mysql> DESC CLAIM_SETTLEMENT;
+-------------------+---------------+------+-----+---------+-------+
| Field             | Type          | Null | Key | Default | Extra |
+-------------------+---------------+------+-----+---------+-------+
| settlement_id     | int           | NO   | PRI | NULL    |       |
| claim_id          | int           | NO   | UNI | NULL    |       |
| settlement_date   | date          | YES  |     | NULL    |       |
| settlement_amount | decimal(15,2) | NO   |     | NULL    |       |
| settlement_method | varchar(50)   | YES  |     | NULL    |       |
| settlement_status | varchar(30)   | YES  |     | NULL    |       |
+-------------------+---------------+------+-----+---------+-------+
6 rows in set (0.00 sec)
```

vi)PAYMENT is created and attributes are added to it

```
mysql> CREATE TABLE PAYMENT (
    ->     payment_id INT PRIMARY KEY,
    ->     settlement_id INT NOT NULL,
    ->     payment_date DATE,
    ->     payment_amount DECIMAL(15,2) NOT NULL,
    ->     payment_method VARCHAR(50),
    ->     payment_status VARCHAR(30),
    ->     payer VARCHAR(100),
    ->     payee VARCHAR(100),
    ->
    ->     FOREIGN KEY (settlement_id)
    ->         REFERENCES CLAIM_SETTLEMENT(settlement_id),
    ->
    ->     UNIQUE (settlement_id)
    -> );
Query OK, 0 rows affected (0.11 sec)

mysql> DESC PAYMENT;
+----------------+---------------+------+-----+---------+-------+
| Field          | Type          | Null | Key | Default | Extra |
+----------------+---------------+------+-----+---------+-------+
| payment_id     | int           | NO   | PRI | NULL    |       |
| settlement_id  | int           | NO   | UNI | NULL    |       |
| payment_date   | date          | YES  |     | NULL    |       |
| payment_amount | decimal(15,2) | NO   |     | NULL    |       |
| payment_method | varchar(50)   | YES  |     | NULL    |       |
| payment_status | varchar(30)   | YES  |     | NULL    |       |
| payer          | varchar(100)  | YES  |     | NULL    |       |
| payee          | varchar(100)  | YES  |     | NULL    |       |
+----------------+---------------+------+-----+---------+-------+
8 rows in set (0.01 sec)
```

confirming all 10 tables

```
mysql> SHOW TABLES;
+-----------------------------------+
| Tables_in_insurance_claims_db     |
+-----------------------------------+
| assessor                          |
| claim                             |
| claim_assessment                  |
| claim_settlement                  |
| customer                          |
| incident                          |
| insurance_policy                  |
| insured_asset                     |
| payment                           |
| policy_type                       |
+-----------------------------------+
10 rows in set (0.00 sec)

mysql>
```

1)ALTER command

```
mysql> DESCRIBE CLAIM;
+-------------------+---------------+------+-----+---------+-------+
| Field             | Type          | Null | Key | Default | Extra |
+-------------------+---------------+------+-----+---------+-------+
| claim_id          | int           | NO   | PRI | NULL    |       |
| policy_id         | int           | NO   | MUL | NULL    |       |
| incident_id       | int           | NO   | UNI | NULL    |       |
| claim_date        | date          | NO   |     | NULL    |       |
| claimed_amount    | decimal(15,2) | NO   |     | NULL    |       |
| claim_status      | varchar(30)   | NO   |     | NULL    |       |
| claim_description | varchar(500)  | YES  |     | NULL    |       |
| phase1_test_note  | varchar(100)  | YES  |     | NULL    |       |
+-------------------+---------------+------+-----+---------+-------+
8 rows in set (0.00 sec)

mysql> ALTER TABLE CLAIM
    -> DROP COLUMN phase1_test_note;
Query OK, 0 rows affected (0.06 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> DESCRIBE CLAIM;
+-------------------+---------------+------+-----+---------+-------+
| Field             | Type          | Null | Key | Default | Extra |
+-------------------+---------------+------+-----+---------+-------+
| claim_id          | int           | NO   | PRI | NULL    |       |
| policy_id         | int           | NO   | MUL | NULL    |       |
| incident_id       | int           | NO   | UNI | NULL    |       |
| claim_date        | date          | NO   |     | NULL    |       |
| claimed_amount    | decimal(15,2) | NO   |     | NULL    |       |
| claim_status      | varchar(30)   | NO   |     | NULL    |       |
| claim_description | varchar(500)  | YES  |     | NULL    |       |
+-------------------+---------------+------+-----+---------+-------+
7 rows in set (0.16 sec)

mysql> USE insurance_claims_db;
Database changed
mysql>
mysql> ALTER TABLE CLAIM
    -> ADD COLUMN claim_priority
    -> ENUM('Low', 'Medium', 'High', 'Critical')
    -> DEFAULT 'Medium';
Query OK, 0 rows affected (0.09 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> DESCRIBE CLAIM;
+-------------------+-----------------------------------------+------+-----+---------+-------+
| Field             | Type                                    | Null | Key | Default | Extra |
+-------------------+-----------------------------------------+------+-----+---------+-------+
| claim_id          | int                                     | NO   | PRI | NULL    |       |
| policy_id         | int                                     | NO   | MUL | NULL    |       |
| incident_id       | int                                     | NO   | UNI | NULL    |       |
| claim_date        | date                                    | NO   |     | NULL    |       |
| claimed_amount    | decimal(15,2)                           | NO   |     | NULL    |       |
| claim_status      | varchar(30)                             | NO   |     | NULL    |       |
| claim_description | varchar(500)                            | YES  |     | NULL    |       |
| claim_priority    | enum('Low','Medium','High','Critical') | YES  |     | Medium  |       |
+-------------------+-----------------------------------------+------+-----+---------+-------+
8 rows in set (0.00 sec)
```

The ALTER TABLE command was used to introduce a claim_priority attribute with Low, Medium, High and Critical classifications to support urgency-based claim processing.

2)DROP : context (insurance companies temporarily stage imported claim records before validating and transferring them into the main database, here we show removal of an obsolete staging table)

```
mysql> CREATE TABLE claim_import_staging (
    ->     import_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     claim_reference VARCHAR(30),
    ->     import_date DATE,
    ->     validation_status VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql> SHOW TABLES LIKE 'claim_import_staging';
+-----------------------------------------------------+
| Tables_in_insurance_claims_db (claim_import_staging) |
+-----------------------------------------------------+
| claim_import_staging                                |
+-----------------------------------------------------+
1 row in set (0.01 sec)

mysql> DROP TABLE claim_import_staging;
Query OK, 0 rows affected (0.06 sec)

mysql> SHOW TABLES LIKE 'claim_import_staging';
Empty set (0.01 sec)

mysql>
```

first , created the staging table, then verified its presence , dropped it and verified the drop again

hence , removing an obsolete claim import staging table after its purpose of temporarily holding imported claim records was completed

6. DML STATEMENTS

Data Manipulation Language (DML) statements are used to insert, retrieve, update and delete records.

The project demonstrates:

- INSERT – Adds records to database tables.
- SELECT – Retrieves stored information.
- UPDATE – Modifies existing records.
- DELETE – Removes specified records.

Sample records are inserted into the ten tables to demonstrate the relationships and database operations.

A temporary staging table is also used to demonstrate insertion, modification and deletion of records.

1) adding sample data to the database using INSERT and performing SELECT queries
i) adding sample data to all 10 tables

```
mysql> INSERT INTO CUSTOMER
    -> (customer_id, name, date_of_birth, gender, phone, email, address)
    -> VALUES
    -> (1, 'Shreya Shenoy', '2004-06-15', 'Female', '9876543210', 'shreya@gmail.com', 'Bangalore'),
    -> (2, 'Shreya Patil', '1998-11-22', 'female', '9876501234', 'patil@gmail.com', 'Mumbai'),
    -> (3, 'Shreya Mattimadu', '1995-03-10', 'Female', '9988776655', 'boss@gmail.com', 'Chennai'),
    -> (4, 'Shreelakshmi Bhat', '1989-08-25', 'Female', '9123456789', 'shili@gmail.com', 'Hyderabad'),
    -> (5, 'Lilas Ikuta', '1992-12-05', 'Female', '9012345678', 'yoasobi@gmail.com', 'Kochi');
Query OK, 5 rows affected (0.04 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> INSERT INTO POLICY_TYPE
    -> (policy_type_id, policy_name, description, base_premium, default_coverage_amount)
    -> VALUES
    -> (101, 'Car Insurance', 'Insurance coverage for vehicles', 15000.00, 1000000.00),
    -> (102, 'Health Insurance', 'Medical and hospitalization coverage', 20000.00, 5000000.00),
    -> (103, 'Home Insurance', 'Protection against property damage', 12000.00, 8000000.00),
    -> (104, 'Travel Insurance', 'Coverage for travel related risks', 5000.00, 1000000.00),
    -> (105, 'Life Insurance', 'Financial protection for policyholder', 30000.00, 10000000.00);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> INSERT INTO INCIDENT
    -> (incident_id, incident_type, incident_date, incident_time, location, description, reported_date)
    -> VALUES
    -> (201, 'Car Accident', '2026-08-20', '18:30:00', 'Bangalore', 'Collision with another vehicle', '2026-08-21'),
    -> (202, 'Hospitalization', '2026-07-15', '10:00:00', 'Chennai', 'Hospitalization due to illness', '2026-07-16'),
    -> (203, 'Property Damage', '2026-06-10', '14:30:00', 'Mumbai', 'Damage to house due to heavy rainfall', '2026-06-11'),
    -> (204, 'Car Theft', '2026-08-05', '22:15:00', 'Hyderabad', 'Vehicle reported stolen from parking area', '2026-08-06'),
    -> (205, 'Medical Emergency', '2026-05-18', '09:45:00', 'Kochi', 'Emergency medical treatment required', '2026-05-18'),
    -> (206, 'Travel Cancellation', '2026-07-28', '06:30:00', 'Delhi', 'Flight cancelled due to operational issues', '2026-07-29');
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO ASSESSOR
    ->
    -> (assessor_id, name, contact, specialization, experience, status)
    ->
    -> VALUES
    ->
    -> (301, 'Chinaswamy', '9877001122', 'Motor Claims', 8, 'Active'),
    ->
    -> (302, 'Muthuswamy', '9877002233', 'Health Claims', 6, 'Active'),
    ->
    -> (303, 'Bahubali', '9877003344', 'Property Claims', 10, 'Active'),
    ->
    -> (304, 'Preeti', '9877004455', 'Travel Claims', 5, 'Active');
Query OK, 4 rows affected (0.02 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO INSURANCE_POLICY
    -> (policy_id, customer_id, policy_type_id, start_date, end_date,
    ->  premium_amount, coverage_amount, policy_status)
    -> VALUES
    -> (1001, 1, 101, '2026-01-01', '2026-12-31', 18000.00, 1000000.00, 'Active'),
    -> (1002, 1, 102, '2026-01-01', '2026-12-31', 22000.00, 5000000.00, 'Active'),
    -> (1003, 2, 103, '2026-02-01', '2027-01-31', 14000.00, 8000000.00, 'Active'),
    -> (1004, 3, 102, '2026-01-15', '2027-01-14', 25000.00, 5000000.00, 'Active'),
    -> (1005, 4, 101, '2026-03-01', '2027-02-28', 17000.00, 1200000.00, 'Active'),
    -> (1006, 5, 104, '2026-04-01', '2027-03-31', 6000.00, 1000000.00, 'Active');
Query OK, 6 rows affected (0.02 sec)
Records: 6  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO INSURED_ASSET
    -> (asset_id, policy_id, asset_type, asset_description,
    ->  asset_value, registration_or_reference_no, location)
    -> VALUES
    -> (401, 1001, 'Car', 'Hyundai Creta', 1500000.00, 'KA01AB1234', 'Bangalore'),
    -> (402, 1003, 'House', 'Residential Apartment', 7500000.00, 'PROP-MUM-003', 'Mumbai'),
    -> (403, 1005, 'Car', 'Honda City', 1300000.00, 'TS09CD5678', 'Hyderabad'),
    -> (404, 1002, 'Health', 'Health Insurance Coverage', 0.00, 'HEALTH-1002', 'Bangalore'),
    -> (405, 1004, 'Health', 'Health Insurance Coverage', 0.00, 'HEALTH-1004', 'Chennai'),
    -> (406, 1006, 'Travel', 'International Travel Coverage', 0.00, 'TRAVEL-1006', 'Kochi');
Query OK, 6 rows affected (0.02 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> INSERT INTO CLAIM
    -> (claim_id, policy_id, incident_id, claim_date,
    ->  claimed_amount, claim_status, claim_description)
    -> VALUES
    -> (501, 1001, 201, '2026-08-21', 200000.00, 'Settled', 'Claim for vehicle accident damage'),
    -> (502, 1004, 202, '2026-07-16', 350000.00, 'Settled', 'Hospitalization expenses'),
    -> (503, 1003, 203, '2026-06-11', 500000.00, 'Settled', 'House damage due to rainfall'),
    -> (504, 1005, 204, '2026-08-06', 900000.00, 'Under Assessment', 'Vehicle theft claim'),
    -> (505, 1002, 205, '2026-05-18', 150000.00, 'Approved', 'Emergency medical treatment'),
    -> (506, 1006, 206, '2026-07-29', 80000.00, 'Under Review', 'Travel cancellation claim');
Query OK, 6 rows affected (0.02 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> INSERT INTO CLAIM_ASSESSMENT
    -> (assessment_id, claim_id, assessor_id, assessment_date,
    ->  assessed_amount, remarks, recommendation)
    -> VALUES
    -> (601, 501, 301, '2026-08-23', 160000.00, 'Vehicle damage verified', 'Approve partially'),
    -> (602, 502, 302, '2026-07-20', 300000.00, 'Hospital bills verified', 'Approve'),
    -> (603, 503, 303, '2026-06-15', 420000.00, 'Property damage inspected', 'Approve partially'),
    -> (604, 504, 301, '2026-08-10', 850000.00, 'Theft documents under verification', 'Pending');
Query OK, 4 rows affected (0.02 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO CLAIM_SETTLEMENT
    -> (settlement_id, claim_id, settlement_date,
    ->  settlement_amount, settlement_method, settlement_status)
    -> VALUES
    -> (701, 501, '2026-08-25', 150000.00, 'Bank Transfer', 'Completed'),
    -> (702, 502, '2026-07-23', 290000.00, 'Bank Transfer', 'Completed'),
    -> (703, 503, '2026-06-20', 400000.00, 'Cheque', 'Completed');INSERT INTO PAYMENT
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

    -> (payment_id, settlement_id, payment_date,
    ->  payment_amount, payment_method, payment_status, payer, payee)
    -> VALUES
    -> (801, 701, '2026-08-26', 150000.00, 'Bank Transfer', 'Completed', 'ABC Insurance', 'Shreya Shenoy'),
    -> (802, 702, '2026-07-24', 290000.00, 'Bank Transfer', 'Completed', 'ABC Insurance', 'Ananya Iyer'),
    -> (803, 703, '2026-06-22', 400000.00, 'Cheque', 'Completed', 'ABC Insurance', 'Rahul Sharma');
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

ALL THE POPULATED VALUES ARE :

```
mysql> SELECT * FROM CUSTOMER;
+-------------+-----------------+---------------+--------+------------+-------------------+-----------+
| customer_id | name            | date_of_birth | gender | phone      | email             | address   |
+-------------+-----------------+---------------+--------+------------+-------------------+-----------+
|           1 | Shreya Shenoy   | 2004-06-15    | Female | 9876543210 | shreya@gmail.com  | Bangalore |
|           2 | Shreya Patil    | 1998-11-22    | female | 9876501234 | patil@gmail.com   | Mumbai    |
|           3 | Shreya Mattimadu| 1995-03-10    | Female | 9988776655 | boss@gmail.com    | Chennai   |
|           4 | Shreelakshmi Bhat| 1989-08-25   | Female | 9123456789 | shili@gmail.com   | Hyderabad |
|           5 | Lilas Ikuta     | 1992-12-05    | Female | 9012345678 | yoasobi@gmail.com | Kochi     |
+-------------+-----------------+---------------+--------+------------+-------------------+-----------+
5 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM POLICY_TYPE;
+----------------+------------------+---------------------------------------+--------------+--------------------------+
| policy_type_id | policy_name      | description                           | base_premium | default_coverage_amount |
+----------------+------------------+---------------------------------------+--------------+--------------------------+
|            101 | Car Insurance    | Insurance coverage for vehicles       |     15000.00 |              1000000.00 |
|            102 | Health Insurance | Medical and hospitalization coverage  |     20000.00 |              5000000.00 |
|            103 | Home Insurance   | Protection against property damage    |     12000.00 |              8000000.00 |
|            104 | Travel Insurance | Coverage for travel related risks     |      5000.00 |              1000000.00 |
|            105 | Life Insurance   | Financial protection for policyholder |     30000.00 |             10000000.00 |
+----------------+------------------+---------------------------------------+--------------+--------------------------+
5 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM INSURANCE_POLICY;
+-----------+-------------+----------------+------------+------------+----------------+-----------------+---------------+
| policy_id | customer_id | policy_type_id | start_date | end_date   | premium_amount | coverage_amount | policy_status |
+-----------+-------------+----------------+------------+------------+----------------+-----------------+---------------+
|      1001 |           1 |            101 | 2026-01-01 | 2026-12-31 |       18000.00 |      1000000.00 | Active        |
|      1002 |           1 |            102 | 2026-01-01 | 2026-12-31 |       22000.00 |      5000000.00 | Active        |
|      1003 |           2 |            103 | 2026-02-01 | 2027-01-31 |       14000.00 |      8000000.00 | Active        |
|      1004 |           3 |            102 | 2026-01-15 | 2027-01-14 |       25000.00 |      5000000.00 | Active        |
|      1005 |           4 |            101 | 2026-03-01 | 2027-02-28 |       17000.00 |      1200000.00 | Active        |
|      1006 |           5 |            104 | 2026-04-01 | 2027-03-31 |        6000.00 |      1000000.00 | Active        |
+-----------+-------------+----------------+------------+------------+----------------+-----------------+---------------+
6 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM INSURED_ASSET;
+----------+-----------+------------+-----------------------------+-------------+-----------------------------+-----------+
| asset_id | policy_id | asset_type | asset_description           | asset_value | registration_or_reference_no | location  |
+----------+-----------+------------+-----------------------------+-------------+-----------------------------+-----------+
|      401 |      1001 | Car        | Hyundai Creta               |  1500000.00 | KA01AB1234                  | Bangalore |
|      402 |      1003 | House      | Residential Apartment       |  7500000.00 | PROP-MUM-003                | Mumbai    |
|      403 |      1005 | Car        | Honda City                  |  1300000.00 | TS09CD5678                  | Hyderabad |
|      404 |      1002 | Health     | Health Insurance Coverage   |        0.00 | HEALTH-1002                 | Bangalore |
|      405 |      1004 | Health     | Health Insurance Coverage   |        0.00 | HEALTH-1004                 | Chennai   |
|      406 |      1006 | Travel     | International Travel Coverage |      0.00 | TRAVEL-1006                 | Kochi     |
+----------+-----------+------------+-----------------------------+-------------+-----------------------------+-----------+
6 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM INCIDENT;
+-------------+-------------------+---------------+---------------+-----------+-----------------------------------------+---------------+
| incident_id | incident_type     | incident_date | incident_time | location  | description                             | reported_date |
+-------------+-------------------+---------------+---------------+-----------+-----------------------------------------+---------------+
|         201 | Car Accident      | 2026-08-20    | 18:30:00      | Bangalore | Collision with another vehicle          | 2026-08-21    |
|         202 | Hospitalization   | 2026-07-15    | 10:00:00      | Chennai   | Hospitalization due to illness          | 2026-07-16    |
|         203 | Property Damage   | 2026-06-10    | 14:30:00      | Mumbai    | Damage to house due to heavy rainfall   | 2026-06-11    |
|         204 | Car Theft         | 2026-08-05    | 22:15:00      | Hyderabad | Vehicle reported stolen from parking area | 2026-08-06  |
|         205 | Medical Emergency | 2026-05-18    | 09:45:00      | Kochi     | Emergency medical treatment required    | 2026-05-18    |
|         206 | Travel Cancellation | 2026-07-28  | 06:30:00      | Delhi     | Flight cancelled due to operational issues | 2026-07-29 |
+-------------+-------------------+---------------+---------------+-----------+-----------------------------------------+---------------+
6 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM CLAIM;
+----------+-----------+-------------+------------+----------------+------------------+-----------------------------------+
| claim_id | policy_id | incident_id | claim_date | claimed_amount | claim_status     | claim_description                 |
+----------+-----------+-------------+------------+----------------+------------------+-----------------------------------+
|      501 |      1001 |         201 | 2026-08-21 |      200000.00 | Settled          | Claim for vehicle accident damage |
|      502 |      1004 |         202 | 2026-07-16 |      350000.00 | Settled          | Hospitalization expenses          |
|      503 |      1003 |         203 | 2026-06-11 |      500000.00 | Settled          | House damage due to rainfall      |
|      504 |      1005 |         204 | 2026-08-06 |      900000.00 | Under Assessment | Vehicle theft claim               |
|      505 |      1002 |         205 | 2026-05-18 |      150000.00 | Approved         | Emergency medical treatment       |
|      506 |      1006 |         206 | 2026-07-29 |       80000.00 | Under Review     | Travel cancellation claim         |
+----------+-----------+-------------+------------+----------------+------------------+-----------------------------------+
6 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM ASSESSOR;
+-------------+------------+------------+-----------------+------------+--------+
| assessor_id | name       | contact    | specialization  | experience | status |
+-------------+------------+------------+-----------------+------------+--------+
|         301 | Chinaswamy | 9877001122 | Motor Claims    |          8 | Active |
|         302 | Muthuswamy | 9877002233 | Health Claims   |          6 | Active |
|         303 | Bahubali   | 9877003344 | Property Claims |         10 | Active |
|         304 | Preeti     | 9877004455 | Travel Claims   |          5 | Active |
+-------------+------------+------------+-----------------+------------+--------+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM CLAIM_ASSESSMENT;
+---------------+----------+-------------+-----------------+-----------------+-----------------------------------+-------------------+
| assessment_id | claim_id | assessor_id | assessment_date | assessed_amount | remarks                           | recommendation    |
+---------------+----------+-------------+-----------------+-----------------+-----------------------------------+-------------------+
|           601 |      501 |         301 | 2026-08-23      |       160000.00 | Vehicle damage verified           | Approve partially |
|           602 |      502 |         302 | 2026-07-20      |       300000.00 | Hospital bills verified           | Approve           |
|           603 |      503 |         303 | 2026-06-15      |       420000.00 | Property damage inspected         | Approve partially |
|           604 |      504 |         301 | 2026-08-10      |       850000.00 | Theft documents under verification | Pending           |
+---------------+----------+-------------+-----------------+-----------------+-----------------------------------+-------------------+
4 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM CLAIM_SETTLEMENT;
+---------------+----------+-----------------+-------------------+-------------------+-----------------+
| settlement_id | claim_id | settlement_date | settlement_amount | settlement_method | settlement_status |
+---------------+----------+-----------------+-------------------+-------------------+-----------------+
|           701 |      501 | 2026-08-25      |         150000.00 | Bank Transfer     | Completed       |
|           702 |      502 | 2026-07-23      |         290000.00 | Bank Transfer     | Completed       |
|           703 |      503 | 2026-06-20      |         400000.00 | Cheque            | Completed       |
+---------------+----------+-----------------+-------------------+-------------------+-----------------+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM PAYMENT;
+------------+---------------+--------------+----------------+----------------+----------------+---------------+---------------+
| payment_id | settlement_id | payment_date | payment_amount | payment_method | payment_status | payer         | payee         |
+------------+---------------+--------------+----------------+----------------+----------------+---------------+---------------+
|        801 |           701 | 2026-08-26   |      150000.00 | Bank Transfer  | Completed      | ABC Insurance | Shreya Shenoy |
|        802 |           702 | 2026-07-24   |      290000.00 | Bank Transfer  | Completed      | ABC Insurance | Ananya Iyer   |
|        803 |           703 | 2026-06-22   |      400000.00 | Cheque         | Completed      | ABC Insurance | Rahul Sharma  |
+------------+---------------+--------------+----------------+----------------+----------------+---------------+---------------+
3 rows in set (0.00 sec)
```

3)UPDATE

here we update a customer's address records

we're using test customer

```
mysql> INSERT INTO CUSTOMER
    -> (customer_id, name, date_of_birth, gender, phone, email, address)
    -> VALUES
    -> (9999, 'Test Customer', '2000-01-01', 'Other',
    ->  '9999999999', 'test.customer@example.com', 'Old Address');
Query OK, 1 row affected (0.03 sec)

mysql> INSERT INTO CUSTOMER
    -> (customer_id, name, date_of_birth, gender, phone, email, address)
    -> VALUES
    -> (9999, 'Test Customer', '2000-01-01', 'Other',
    ->  '9999999999', 'test.customer@example.com', 'Old Address');
ERROR 1062 (23000): Duplicate entry '9999' for key 'customer.PRIMARY'
mysql> SELECT *
    -> FROM CUSTOMER
    -> WHERE customer_id = 9999;
+-------------+---------------+---------------+--------+------------+---------------------------+-------------+
| customer_id | name          | date_of_birth | gender | phone      | email                     | address     |
+-------------+---------------+---------------+--------+------------+---------------------------+-------------+
|        9999 | Test Customer | 2000-01-01    | Other  | 9999999999 | test.customer@example.com | Old Address |
+-------------+---------------+---------------+--------+------------+---------------------------+-------------+
1 row in set (0.01 sec)

mysql> UPDATE CUSTOMER
    -> SET address = 'New Address, Bengaluru'
    -> WHERE customer_id = 9999;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT customer_id, name, address
    -> FROM CUSTOMER
    -> WHERE customer_id = 9999;
+-------------+---------------+------------------------+
| customer_id | name          | address                |
+-------------+---------------+------------------------+
|        9999 | Test Customer | New Address, Bengaluru |
+-------------+---------------+------------------------+
1 row in set (0.00 sec)

mysql>
```

here, the UPDATE statement was used to modify the address of a customer record, demonstrating how customer information can be maintained using a conditional WHERE clause.

4. DELETE

```
mysql> DELETE FROM CUSTOMER
    -> WHERE customer_id = 9999;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT *
    -> FROM CUSTOMER
    -> WHERE customer_id = 9999;
Empty set (0.01 sec)

mysql>
```

using the DELETE statement , deleted the test customer and their readings . this is done using its unique customer ID, demonstrating controlled deletion of records without affecting existing customer data

7. QUERIES

7.1 SIMPLE QUERY WITH GROUP BY, AGGREGATE

ii)Using concepts of JOIN , GROUP BY , COUNT and HAVING to find customers with multiple policies (using aggregation)

```
mysql> SELECT
    ->     c.customer_id,
    ->     c.name,
    ->     COUNT(p.policy_id) AS number_of_policies
    -> FROM CUSTOMER c
    -> JOIN INSURANCE_POLICY p
    ->     ON c.customer_id = p.customer_id
    -> GROUP BY c.customer_id, c.name
    -> HAVING COUNT(p.policy_id) > 1;
+-------------+---------------+--------------------+
| customer_id | name          | number_of_policies |
+-------------+---------------+--------------------+
|           1 | Shreya Shenoy |                  2 |
+-------------+---------------+--------------------+
1 row in set (0.02 sec)

mysql>
```

ii) Using concepts of aggregate AVG , JOIN and GROUP BY to find average claim amount by policy type

```
mysql> SELECT
    ->     pt.policy_name,
    ->     AVG(cl.claimed_amount) AS average_claim_amount
    -> FROM POLICY_TYPE pt
    -> JOIN INSURANCE_POLICY p
    ->     ON pt.policy_type_id = p.policy_type_id
    -> JOIN CLAIM cl
    ->     ON p.policy_id = cl.policy_id
    -> GROUP BY pt.policy_type_id, pt.policy_name;
+------------------+----------------------+
| policy_name      | average_claim_amount |
+------------------+----------------------+
| Car Insurance    |         550000.000000 |
| Health Insurance |         250000.000000 |
| Home Insurance   |         500000.000000 |
| Travel Insurance |          80000.000000 |
+------------------+----------------------+
4 rows in set (0.04 sec)

mysql>
```

7.2 UPDATE OPERATION

here we update a customer's address records

we're using test customer

```
mysql> INSERT INTO CUSTOMER
    -> (customer_id, name, date_of_birth, gender, phone, email, address)
    -> VALUES
    -> (9999, 'Test Customer', '2000-01-01', 'Other',
    ->  '9999999999', 'test.customer@example.com', 'Old Address');
Query OK, 1 row affected (0.03 sec)

mysql> INSERT INTO CUSTOMER
    -> (customer_id, name, date_of_birth, gender, phone, email, address)
    -> VALUES
    -> (9999, 'Test Customer', '2000-01-01', 'Other',
    ->  '9999999999', 'test.customer@example.com', 'Old Address');
ERROR 1062 (23000): Duplicate entry '9999' for key 'customer.PRIMARY'
mysql> SELECT *
    -> FROM CUSTOMER
    -> WHERE customer_id = 9999;
+-------------+---------------+---------------+--------+------------+--------------------------+-------------+
| customer_id | name          | date_of_birth | gender | phone      | email                    | address     |
+-------------+---------------+---------------+--------+------------+--------------------------+-------------+
|        9999 | Test Customer | 2000-01-01    | Other  | 9999999999 | test.customer@example.com | Old Address |
+-------------+---------------+---------------+--------+------------+--------------------------+-------------+
1 row in set (0.01 sec)

mysql> UPDATE CUSTOMER
    -> SET address = 'New Address, Bengaluru'
    -> WHERE customer_id = 9999;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT customer_id, name, address
    -> FROM CUSTOMER
    -> WHERE customer_id = 9999;
+-------------+---------------+------------------------+
| customer_id | name          | address                |
+-------------+---------------+------------------------+
|        9999 | Test Customer | New Address, Bengaluru |
+-------------+---------------+------------------------+
1 row in set (0.00 sec)

mysql>
```

here, the UPDATE statement was used to modify the address of a customer record, demonstrating how customer information can be maintained using a conditional WHERE clause.

7.3 DELETE OPERATION

```
mysql> DELETE FROM CUSTOMER
    -> WHERE customer_id = 9999;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT *
    -> FROM CUSTOMER
    -> WHERE customer_id = 9999;
Empty set (0.01 sec)

mysql>
```

using the DELETE statement , deleted the test customer and their readings . this is done using its unique customer ID, demonstrating controlled deletion of records without affecting existing customer data

7.4 CORRELATED QUERY

```
mysql> SELECT
    ->     cl.claim_id,
    ->     cl.policy_id,
    ->     cl.claimed_amount
    -> FROM CLAIM cl
    -> WHERE EXISTS (
    ->     SELECT 1
    ->     FROM INSURANCE_POLICY ip
    ->     WHERE ip.policy_id = cl.policy_id
    -> );
+----------+-----------+----------------+
| claim_id | policy_id | claimed_amount |
+----------+-----------+----------------+
|      501 |      1001 |      200000.00 |
|      505 |      1002 |      150000.00 |
|      503 |      1003 |      500000.00 |
|      502 |      1004 |      350000.00 |
|      504 |      1005 |      900000.00 |
|      506 |      1006 |       80000.00 |
+----------+-----------+----------------+
6 rows in set (0.05 sec)

mysql> |
```

A correlated subquery uses the outer query's policy ID to identify claims associated with existing insurance policies.

7.5 NESTED QUERY

```
mysql> SELECT
    ->     claim_id,
    ->     policy_id,
    ->     claimed_amount,
    ->     claim_status
    -> FROM CLAIM
    -> WHERE claimed_amount > (
    ->     SELECT AVG(claimed_amount)
    ->     FROM CLAIM
    -> );
+----------+-----------+----------------+------------------+
| claim_id | policy_id | claimed_amount | claim_status     |
+----------+-----------+----------------+------------------+
|      503 |      1003 |      500000.00 | Settled          |
|      504 |      1005 |      900000.00 | Under Assessment |
+----------+-----------+----------------+------------------+
2 rows in set (0.00 sec)

mysql> |
```

A nested query retrieves claims whose claimed amount exceeds the overall average claim amount.

8. STORED PROCEDURES, FUNCTIONS AND TRIGGERS

8.1 STORED PROCEDURES OR FUNCTIONS

using Procedure to get complete claim details about a user by just their ID

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetClaimDetails(IN p_claim_id INT)
    -> BEGIN
    ->     SELECT
    ->         c.name AS customer_name,
    ->         p.policy_id,
    ->         pt.policy_name,
    ->         ia.asset_type,
    ->         i.incident_type,
    ->         cl.claim_id,
    ->         cl.claim_date,
    ->         cl.claimed_amount,
    ->         cl.claim_status,
    ->         ca.assessed_amount,
    ->         cs.settlement_amount,
    ->         pay.payment_amount
    ->     FROM CLAIM cl
    ->     JOIN INSURANCE_POLICY p
    ->         ON cl.policy_id = p.policy_id
    ->     JOIN CUSTOMER c
    ->         ON p.customer_id = c.customer_id
    ->     JOIN POLICY_TYPE pt
    ->         ON p.policy_type_id = pt.policy_type_id
    ->     JOIN INSURED_ASSET ia
    ->         ON p.policy_id = ia.policy_id
    ->     JOIN INCIDENT i
    ->         ON cl.incident_id = i.incident_id
    ->     LEFT JOIN CLAIM_ASSESSMENT ca
    ->         ON cl.claim_id = ca.claim_id
    ->     LEFT JOIN CLAIM_SETTLEMENT cs
    ->         ON cl.claim_id = cs.claim_id
    ->     LEFT JOIN PAYMENT pay
    ->         ON cs.settlement_id = pay.settlement_id
    ->     WHERE cl.claim_id = p_claim_id;
    -> END //
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> DELIMITER ;
mysql> CALL GetClaimDetails(501);
+---------------+-----------+--------------+------------+---------------+----------+------------+---------------+--------------+-----------------+-------------------+----------------+
| customer_name | policy_id | policy_name  | asset_type | incident_type | claim_id | claim_date | claimed_amount | claim_status | assessed_amount | settlement_amount | payment_amount |
+---------------+-----------+--------------+------------+---------------+----------+------------+---------------+--------------+-----------------+-------------------+----------------+
| Shreya Shenoy |      1001 | Car Insurance | Car        | Car Accident  |      501 | 2026-08-21 |      200000.00 | Settled      |       160000.00 |         150000.00 |      150000.00 |
+---------------+-----------+--------------+------------+---------------+----------+------------+---------------+--------------+-----------------+-------------------+----------------+
1 row in set (0.20 sec)

Query OK, 0 rows affected (0.25 sec)
```

8.2 TRIGGERS

using a trigger function to automatically update claim status after settlement

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER after_claim_settlement
    -> AFTER INSERT ON CLAIM_SETTLEMENT
    -> FOR EACH ROW
    -> BEGIN
    ->     UPDATE CLAIM
    ->     SET claim_status = 'Settled'
    ->     WHERE claim_id = NEW.claim_id;
    -> END //
Query OK, 0 rows affected (0.08 sec)

mysql>
mysql> DELIMITER ;
mysql>
```

checking claim's current status

```
mysql> DELIMITER ;
mysql> SELECT claim_id, claim_status
    -> FROM CLAIM
    -> WHERE claim_id = 501;
+------------+----------------+
| claim_id | claim_status |
+------------+----------------+
|       501 | Settled        |
+------------+----------------+
1 row in set (0.01 sec)

mysql>
```

8.3 FUNCTIONS

Calculate settlement percentage using the concept of settlement amount ÷ claimed amount × 100 by creating a function

```
mysql> DELIMITER //
mysql>
mysql> CREATE FUNCTION CalculateSettlementPercentage(p_claim_id INT)
    -> RETURNS DECIMAL(5,2)
    -> DETERMINISTIC
    -> READS SQL DATA
    -> BEGIN
    ->     DECLARE v_claimed_amount DECIMAL(15,2);
    ->     DECLARE v_settlement_amount DECIMAL(15,2);
    ->
    ->     SELECT
    ->         cl.claimed_amount,
    ->         cs.settlement_amount
    ->     INTO
    ->         v_claimed_amount,
    ->         v_settlement_amount
    ->     FROM CLAIM cl
    ->     LEFT JOIN CLAIM_SETTLEMENT cs
    ->         ON cl.claim_id = cs.claim_id
    ->     WHERE cl.claim_id = p_claim_id;
    ->
    ->     IF v_claimed_amount IS NULL
    ->        OR v_settlement_amount IS NULL THEN
    ->         RETURN NULL;
    ->     END IF;
    ->
    ->     RETURN ROUND(
    ->         (v_settlement_amount / v_claimed_amount) * 100,
    ->         2
    ->     );
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT CalculateSettlementPercentage(501);
+-------------------------------------+
| CalculateSettlementPercentage(501) |
+-------------------------------------+
|                              75.00 |
+-------------------------------------+
1 row in set (0.00 sec)

mysql> SELECT CalculateSettlementPercentage(502);
+-------------------------------------+
| CalculateSettlementPercentage(502) |
+-------------------------------------+
|                              82.86 |
+-------------------------------------+
1 row in set (0.01 sec)

mysql>
```

9. Other Miscellaneous Queries we executed

9.1 GRANTING ACCESS TO TEAMMATE

database created by one teammate and all access being granted to the other

```
mysql> CREATE USER 'shreyap'@'%' IDENTIFIED BY 'shreyap170606';
Query OK, 0 rows affected (0.14 sec)

mysql> GRANT ALL PRIVILEGES
    -> ON insurance_claims_db.*
    -> TO 'shreyap'@'%';
Query OK, 0 rows affected (0.03 sec)

mysql> SHOW GRANTS FOR 'shreyap'@'%';
+-------------------------------------------------------------------+
| Grants for shreyap@%                                              |
+-------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `shreyap`@`%`                               |
| GRANT ALL PRIVILEGES ON `insurance_claims_db`.* TO `shreyap`@`%` |
+-------------------------------------------------------------------+
2 rows in set (0.01 sec)

mysql>
```

Able to connect to the database using ip address and port number. Both systems need to be on the same network to do so.

```
C:\Users\shrey>"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -h 10.30.95.233 -P 3306 -u shreyap -p
Enter password: *************
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show tables;
ERROR 1046 (3D000): No database selected
mysql> use insurance_claims_db;
Database changed
mysql> show tables;
+-------------------------------+
| Tables_in_insurance_claims_db |
+-------------------------------+
| assessor                      |
| claim                         |
| claim_assessment              |
| claim_settlement              |
| customer                      |
| incident                      |
| insurance_policy              |
| insured_asset                 |
| payment                       |
| policy_type                   |
+-------------------------------+
10 rows in set (0.07 sec)
```

Tasks Queries :

Teammate 1: Shreya Shenoy

1)using INNER JOIN and multiple table JOIN concept ,display policies with customer and policy type

```
mysql> SELECT
    ->     p.policy_id,
    ->     c.name AS customer_name,
    ->     pt.policy_name,
    ->     p.start_date,
    ->     p.end_date,
    ->     p.premium_amount,
    ->     p.coverage_amount,
    ->     p.policy_status
    -> FROM INSURANCE_POLICY p
    -> JOIN CUSTOMER c
    ->     ON p.customer_id = c.customer_id
    -> JOIN POLICY_TYPE pt
    ->     ON p.policy_type_id = pt.policy_type_id;
+-----------+------------------+------------------+------------+------------+----------------+-----------------+---------------+
| policy_id | customer_name    | policy_name      | start_date | end_date   | premium_amount | coverage_amount | policy_status |
+-----------+------------------+------------------+------------+------------+----------------+-----------------+---------------+
|      1001 | Shreya Shenoy    | Car Insurance    | 2026-01-01 | 2026-12-31 |       18000.00 |      1000000.00 | Active        |
|      1002 | Shreya Shenoy    | Health Insurance | 2026-01-01 | 2026-12-31 |       22000.00 |      5000000.00 | Active        |
|      1003 | Shreya Patil     | Home Insurance   | 2026-02-01 | 2027-01-31 |       14000.00 |      8000000.00 | Active        |
|      1004 | Shreya Mattimadu | Health Insurance | 2026-01-15 | 2027-01-14 |       25000.00 |      5000000.00 | Active        |
|      1005 | Shreelakshmi Bhat | Car Insurance   | 2026-03-01 | 2027-02-28 |       17000.00 |      1200000.00 | Active        |
|      1006 | Lilas Ikuta      | Travel Insurance | 2026-04-01 | 2027-03-31 |        6000.00 |      1000000.00 | Active        |
+-----------+------------------+------------------+------------+------------+----------------+-----------------+---------------+
6 rows in set (0.17 sec)

mysql>
```

this retrieves insurance policies along with the corresponding customer and policy type details using INNER JOINs.

2)Using concepts of JOIN , GROUP BY , COUNT and HAVING to find customers with multiple policies (using aggregation)

```
mysql> SELECT
    ->     c.customer_id,
    ->     c.name,
    ->     COUNT(p.policy_id) AS number_of_policies
    -> FROM CUSTOMER c
    -> JOIN INSURANCE_POLICY p
    ->     ON c.customer_id = p.customer_id
    -> GROUP BY c.customer_id, c.name
    -> HAVING COUNT(p.policy_id) > 1;
+-------------+---------------+--------------------+
| customer_id | name          | number_of_policies |
+-------------+---------------+--------------------+
|           1 | Shreya Shenoy |                  2 |
+-------------+---------------+--------------------+
1 row in set (0.02 sec)

mysql>
```

3) Using concepts of aggregate AVG , JOIN and GROUP BY to find average claim amount by policy type

```
mysql> SELECT
    ->     pt.policy_name,
    ->     AVG(cl.claimed_amount) AS average_claim_amount
    -> FROM POLICY_TYPE pt
    -> JOIN INSURANCE_POLICY p
    ->     ON pt.policy_type_id = p.policy_type_id
    -> JOIN CLAIM cl
    ->     ON p.policy_id = cl.policy_id
    -> GROUP BY pt.policy_type_id, pt.policy_name;
+------------------+----------------------+
| policy_name      | average_claim_amount |
+------------------+----------------------+
| Car Insurance    |         550000.000000 |
| Health Insurance |         250000.000000 |
| Home Insurance   |         500000.000000 |
| Travel Insurance |          80000.000000 |
+------------------+----------------------+
4 rows in set (0.04 sec)

mysql>
```

4) Using JOIN to compare the amount claimed with the amount assessed for each claim and calculate the difference

```
mysql> SELECT
    ->     cl.claim_id,
    ->     cl.claimed_amount,
    ->     ca.assessed_amount,
    ->     (cl.claimed_amount - ca.assessed_amount) AS difference
    -> FROM CLAIM cl
    -> JOIN CLAIM_ASSESSMENT ca
    ->     ON cl.claim_id = ca.claim_id;
+----------+----------------+-----------------+------------+
| claim_id | claimed_amount | assessed_amount | difference |
+----------+----------------+-----------------+------------+
|      501 |      200000.00 |       160000.00 |   40000.00 |
|      502 |      350000.00 |       300000.00 |   50000.00 |
|      503 |      500000.00 |       420000.00 |   80000.00 |
|      504 |      900000.00 |       850000.00 |   50000.00 |
+----------+----------------+-----------------+------------+
4 rows in set (0.04 sec)

mysql>
```

5) Making subqueries and using AVG aggregate function to find claims above the average claim amount

```
mysql> SELECT
    ->     claim_id,
    ->     policy_id,
    ->     claimed_amount,
    ->     claim_status
    -> FROM CLAIM
    -> WHERE claimed_amount > (
    ->     SELECT AVG(claimed_amount)
    ->     FROM CLAIM
    -> );
+----------+-----------+----------------+------------------+
| claim_id | policy_id | claimed_amount | claim_status     |
+----------+-----------+----------------+------------------+
|      503 |      1003 |      500000.00 | Settled          |
|      504 |      1005 |      900000.00 | Under Assessment |
+----------+-----------+----------------+------------------+
2 rows in set (0.01 sec)

mysql>
```

6)Using JOIN , and arithmetic functions like ROUND to calculate the percentage of the claimed amount that was ultimately settled for each settled claim

```
mysql> SELECT
    ->     cl.claim_id,
    ->     cl.claimed_amount,
    ->     cs.settlement_amount,
    ->     ROUND(
    ->         (cs.settlement_amount / cl.claimed_amount) * 100,
    ->         2
    ->     ) AS settlement_percentage
    -> FROM CLAIM cl
    -> JOIN CLAIM_SETTLEMENT cs
    ->     ON cl.claim_id = cs.claim_id;
+----------+----------------+-------------------+-----------------------+
| claim_id | claimed_amount | settlement_amount | settlement_percentage |
+----------+----------------+-------------------+-----------------------+
|      501 |      200000.00 |         150000.00 |                 75.00 |
|      502 |      350000.00 |         290000.00 |                 82.86 |
|      503 |      500000.00 |         400000.00 |                 80.00 |
+----------+----------------+-------------------+-----------------------+
3 rows in set (0.05 sec)

mysql>
```

7)using Procedure to get complete claim details about a user by just their ID

```
mysql> DELIMITER //
mysql>
mysql> CREATE PROCEDURE GetClaimDetails(IN p_claim_id INT)
    -> BEGIN
    ->     SELECT
    ->         c.name AS customer_name,
    ->         p.policy_id,
    ->         pt.policy_name,
    ->         ia.asset_type,
    ->         i.incident_type,
    ->         cl.claim_id,
    ->         cl.claim_date,
    ->         cl.claimed_amount,
    ->         cl.claim_status,
    ->         ca.assessed_amount,
    ->         cs.settlement_amount,
    ->         pay.payment_amount
    ->     FROM CLAIM cl
    ->     JOIN INSURANCE_POLICY p
    ->         ON cl.policy_id = p.policy_id
    ->     JOIN CUSTOMER c
    ->         ON p.customer_id = c.customer_id
    ->     JOIN POLICY_TYPE pt
    ->         ON p.policy_type_id = pt.policy_type_id
    ->     JOIN INSURED_ASSET ia
    ->         ON p.policy_id = ia.policy_id
    ->     JOIN INCIDENT i
    ->         ON cl.incident_id = i.incident_id
    ->     LEFT JOIN CLAIM_ASSESSMENT ca
    ->         ON cl.claim_id = ca.claim_id
    ->     LEFT JOIN CLAIM_SETTLEMENT cs
    ->         ON cl.claim_id = cs.claim_id
    ->     LEFT JOIN PAYMENT pay
    ->         ON cs.settlement_id = pay.settlement_id
    ->     WHERE cl.claim_id = p_claim_id;
    -> END //
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> DELIMITER ;
mysql> CALL GetClaimDetails(501);
+---------------+-----------+---------------+------------+---------------+----------+------------+----------------+--------------+-----------------+-------------------+----------------+
| customer_name | policy_id | policy_name   | asset_type | incident_type | claim_id | claim_date | claimed_amount | claim_status | assessed_amount | settlement_amount | payment_amount |
+---------------+-----------+---------------+------------+---------------+----------+------------+----------------+--------------+-----------------+-------------------+----------------+
| Shreya Shenoy |      1001 | Car Insurance | Car        | Car Accident  |      501 | 2026-08-21 |      200000.00 | Settled      |       160000.00 |         150000.00 |      150000.00 |
+---------------+-----------+---------------+------------+---------------+----------+------------+----------------+--------------+-----------------+-------------------+----------------+
1 row in set (0.20 sec)

Query OK, 0 rows affected (0.25 sec)
```

```
mysql> CALL GetClaimDetails(503);
+---------------+-----------+----------------+------------+----------------+----------+------------+----------------+--------------+-----------------+-------------------+----------------+
| customer_name | policy_id | policy_name    | asset_type | incident_type  | claim_id | claim_date | claimed_amount | claim_status | assessed_amount | settlement_amount | payment_amount |
+---------------+-----------+----------------+------------+----------------+----------+------------+----------------+--------------+-----------------+-------------------+----------------+
| Shreya Patil  |      1003 | Home Insurance | House      | Property Damage |      503 | 2026-06-11 |      500000.00 | Settled      |       420000.00 |         400000.00 |      400000.00 |
+---------------+-----------+----------------+------------+----------------+----------+------------+----------------+--------------+-----------------+-------------------+----------------+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)

mysql>
```

Teammate 2: Shreya Patil

Able to connect to the database using ip address and port number. Both systems need to be on the same network to do so.

```
C:\Users\shrey>"C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -h 10.30.95.233 -P 3306 -u shreyap -p
Enter password: *************
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show tables;
ERROR 1046 (3D000): No database selected
mysql> use insurance_claims_db;
Database changed
mysql> show tables;
+------------------------------+
| Tables_in_insurance_claims_db |
+------------------------------+
| assessor                     |
| claim                        |
| claim_assessment             |
| claim_settlement             |
| customer                     |
| incident                     |
| insurance_policy             |
| insured_asset                |
| payment                      |
| policy_type                  |
+------------------------------+
10 rows in set (0.07 sec)
```

1) Finding claims above ₹1,00,000

```
mysql> SELECT
    ->     claim_id,
    ->     policy_id,
    ->     claimed_amount,
    ->     claim_status
    -> FROM CLAIM
    -> WHERE claimed_amount > 100000
    -> ORDER BY claimed_amount DESC;
+----------+-----------+----------------+------------------+
| claim_id | policy_id | claimed_amount | claim_status     |
+----------+-----------+----------------+------------------+
|      504 |      1005 |      900000.00 | Under Assessment |
|      503 |      1003 |      500000.00 | Settled          |
|      502 |      1004 |      350000.00 | Settled          |
|      501 |      1001 |      200000.00 | Settled          |
|      505 |      1002 |      150000.00 | Approved         |
+----------+-----------+----------------+------------------+
5 rows in set (0.24 sec)
```

2)Counting all the claims by status

```
mysql> SELECT
    ->     claim_status,
    ->     COUNT(*) AS number_of_claims
    -> FROM CLAIM
    -> GROUP BY claim_status;
+------------------+------------------+
| claim_status     | number_of_claims |
+------------------+------------------+
| Settled          |                3 |
| Under Assessment |                1 |
| Approved         |                1 |
| Under Review     |                1 |
+------------------+------------------+
4 rows in set (0.02 sec)
```

3)Calculates the total claimed amount for each policy type and orders the results from highest to lowest.

```
mysql> SELECT
    ->     pt.policy_name,
    ->     SUM(cl.claimed_amount) AS total_claimed_amount
    -> FROM POLICY_TYPE pt
    -> JOIN INSURANCE_POLICY p
    ->     ON pt.policy_type_id = p.policy_type_id
    -> JOIN CLAIM cl
    ->     ON p.policy_id = cl.policy_id
    -> GROUP BY pt.policy_type_id, pt.policy_name
    -> ORDER BY total_claimed_amount DESC;
+------------------+----------------------+
| policy_name      | total_claimed_amount |
+------------------+----------------------+
| Car Insurance    |           1100000.00 |
| Health Insurance |            500000.00 |
| Home Insurance   |            500000.00 |
| Travel Insurance |             80000.00 |
+------------------+----------------------+
4 rows in set (0.03 sec)
```

4)Identifies claims where the amount assessed by the assessor is lower than the amount originally claimed.

```
mysql> SELECT
    ->     cl.claim_id,
    ->     cl.claimed_amount,
    ->     ca.assessed_amount
    -> FROM CLAIM cl
    -> JOIN CLAIM_ASSESSMENT ca
    ->     ON cl.claim_id = ca.claim_id
    -> WHERE ca.assessed_amount < cl.claimed_amount;
+----------+----------------+-----------------+
| claim_id | claimed_amount | assessed_amount |
+----------+----------------+-----------------+
|      501 |      200000.00 |       160000.00 |
|      502 |      350000.00 |       300000.00 |
|      503 |      500000.00 |       420000.00 |
|      504 |      900000.00 |       850000.00 |
+----------+----------------+-----------------+
4 rows in set (0.16 sec)
```

5)Identifies claims that do not yet have an associated settlement using a LEFT JOIN and NULL filtering

```
mysql> SELECT
    ->     cl.claim_id,
    ->     cl.policy_id,
    ->     cl.claimed_amount,
    ->     cl.claim_status
    -> FROM CLAIM cl
    -> LEFT JOIN CLAIM_SETTLEMENT cs
    ->     ON cl.claim_id = cs.claim_id
    -> WHERE cs.settlement_id IS NULL;
+----------+-----------+----------------+------------------+
| claim_id | policy_id | claimed_amount | claim_status     |
+----------+-----------+----------------+------------------+
|      504 |      1005 |      900000.00 | Under Assessment |
|      505 |      1002 |      150000.00 | Approved         |
|      506 |      1006 |       80000.00 | Under Review     |
+----------+-----------+----------------+------------------+
3 rows in set (0.15 sec)
```

6)Identifies settlements for which the corresponding payment has not yet been made

```
mysql> SELECT
    ->     cs.settlement_id,
    ->     cs.claim_id,
    ->     cs.settlement_amount,
    ->     cs.settlement_status
    -> FROM CLAIM_SETTLEMENT cs
    -> LEFT JOIN PAYMENT p
    ->     ON cs.settlement_id = p.settlement_id
    -> WHERE p.payment_id IS NULL;
Empty set (0.17 sec)
```

7) Calculate settlement percentage using the concept of settlement amount ÷ claimed amount × 100 by creating a function

```
mysql> DELIMITER //
mysql>
mysql> CREATE FUNCTION CalculateSettlementPercentage(p_claim_id INT)
    -> RETURNS DECIMAL(5,2)
    -> DETERMINISTIC
    -> READS SQL DATA
    -> BEGIN
    ->     DECLARE v_claimed_amount DECIMAL(15,2);
    ->     DECLARE v_settlement_amount DECIMAL(15,2);
    ->
    ->     SELECT
    ->         cl.claimed_amount,
    ->         cs.settlement_amount
    ->     INTO
    ->         v_claimed_amount,
    ->         v_settlement_amount
    ->     FROM CLAIM cl
    ->     LEFT JOIN CLAIM_SETTLEMENT cs
    ->         ON cl.claim_id = cs.claim_id
    ->     WHERE cl.claim_id = p_claim_id;
    ->
    ->     IF v_claimed_amount IS NULL
    ->        OR v_settlement_amount IS NULL THEN
    ->         RETURN NULL;
    ->     END IF;
    ->
    ->     RETURN ROUND(
    ->         (v_settlement_amount / v_claimed_amount) * 100,
    ->         2
    ->     );
    -> END //
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT CalculateSettlementPercentage(501);
+-------------------------------------+
| CalculateSettlementPercentage(501) |
+-------------------------------------+
|                              75.00 |
+-------------------------------------+
1 row in set (0.00 sec)

mysql> SELECT CalculateSettlementPercentage(502);
+-------------------------------------+
| CalculateSettlementPercentage(502) |
+-------------------------------------+
|                              82.86 |
+-------------------------------------+
1 row in set (0.01 sec)

mysql>
```

8) using a trigger function to automatically update claim status after settlement

```
mysql> DELIMITER //
mysql>
mysql> CREATE TRIGGER after_claim_settlement
    -> AFTER INSERT ON CLAIM_SETTLEMENT
    -> FOR EACH ROW
    -> BEGIN
    ->     UPDATE CLAIM
    ->     SET claim_status = 'Settled'
    ->     WHERE claim_id = NEW.claim_id;
    -> END //
Query OK, 0 rows affected (0.08 sec)

mysql>
mysql> DELIMITER ;
mysql>
```

checking claim's current status

```
mysql> DELIMITER ;
mysql> SELECT claim_id, claim_status
    -> FROM CLAIM
    -> WHERE claim_id = 501;
+----------+----------------+
| claim_id | claim_status   |
+----------+----------------+
|      501 | Settled        |
+----------+----------------+
1 row in set (0.01 sec)

mysql>
```

Date: 28-10-2026

PART 2

1. VIEW

here we create a consolidated claim summary

```
mysql> CREATE OR REPLACE VIEW vw_claim_summary AS
    -> SELECT
    ->     cl.claim_id,
    ->     c.customer_id,
    ->     c.name AS customer_name,
    ->     p.policy_id,
    ->     pt.policy_name,
    ->     cl.claim_date,
    ->     cl.claimed_amount,
    ->     cl.claim_status,
    ->     cl.claim_priority
    -> FROM CLAIM cl
    -> JOIN INSURANCE_POLICY p
    ->     ON cl.policy_id = p.policy_id
    -> JOIN CUSTOMER c
    ->     ON p.customer_id = c.customer_id
    -> JOIN POLICY_TYPE pt
    ->     ON p.policy_type_id = pt.policy_type_id;
Query OK, 0 rows affected (0.04 sec)

mysql> SELECT *
    -> FROM vw_claim_summary;
+----------+-------------+------------------+-----------+-----------------+------------+---------------+------------------+----------------+
| claim_id | customer_id | customer_name    | policy_id | policy_name     | claim_date | claimed_amount | claim_status     | claim_priority |
+----------+-------------+------------------+-----------+-----------------+------------+---------------+------------------+----------------+
|      501 |           1 | Shreya Shenoy    |      1001 | Car Insurance   | 2026-08-21 |      200000.00 | Settled          | Medium         |
|      505 |           1 | Shreya Shenoy    |      1002 | Health Insurance | 2026-05-18 |      150000.00 | Approved         | Medium         |
|      503 |           2 | Shreya Patil     |      1003 | Home Insurance  | 2026-06-11 |      500000.00 | Settled          | Medium         |
|      502 |           3 | Shreya Mattimadu |      1004 | Health Insurance | 2026-07-16 |      350000.00 | Settled          | Medium         |
|      504 |           4 | Shreelakshmi Bhat |      1005 | Car Insurance   | 2026-08-06 |      900000.00 | Under Assessment | Medium         |
|      506 |           5 | Lilas Ikuta      |      1006 | Travel Insurance | 2026-07-29 |       80000.00 | Under Review     | Medium         |
+----------+-------------+------------------+-----------+-----------------+------------+---------------+------------------+----------------+
6 rows in set (0.02 sec)

mysql>
```

CREATE VIEW command and its successful execution

```
mysql> SELECT *
    -> FROM vw_claim_summary
    -> WHERE claim_priority IN ('High', 'Critical');
Empty set (0.00 sec)

mysql> SELECT *
    -> FROM vw_claim_summary
    -> WHERE claim_status IS NOT NULL;
+----------+-------------+------------------+-----------+-----------------+------------+---------------+------------------+----------------+
| claim_id | customer_id | customer_name    | policy_id | policy_name     | claim_date | claimed_amount | claim_status     | claim_priority |
+----------+-------------+------------------+-----------+-----------------+------------+---------------+------------------+----------------+
|      501 |           1 | Shreya Shenoy    |      1001 | Car Insurance   | 2026-08-21 |      200000.00 | Settled          | Medium         |
|      505 |           1 | Shreya Shenoy    |      1002 | Health Insurance | 2026-05-18 |      150000.00 | Approved         | Medium         |
|      503 |           2 | Shreya Patil     |      1003 | Home Insurance  | 2026-06-11 |      500000.00 | Settled          | Medium         |
|      502 |           3 | Shreya Mattimadu |      1004 | Health Insurance | 2026-07-16 |      350000.00 | Settled          | Medium         |
|      504 |           4 | Shreelakshmi Bhat |      1005 | Car Insurance   | 2026-08-06 |      900000.00 | Under Assessment | Medium         |
|      506 |           5 | Lilas Ikuta      |      1006 | Travel Insurance | 2026-07-29 |       80000.00 | Under Review     | Medium         |
+----------+-------------+------------------+-----------+-----------------+------------+---------------+------------------+----------------+
6 rows in set (0.01 sec)

mysql>
```

trying to filter the view and selecting only high priority claims , but since existing claims receive medium by default this returns an empty set. therefore, to get an actual result, we can set one existing claim's priority to High after checking its ID, or use a different filter

2.INDEXES

in insurance claims system, staff frequently search and filter claims by their status and date. Indexes help MySQL locate matching records more efficiently

we create a composite index on claim_status and claim_date

EXPLAIN displays the execution plan MySQL considers for the query.

```
mysql> USE insurance_claims_db;
Database changed
mysql>
mysql> CREATE INDEX idx_claim_status_date
    -> ON CLAIM (claim_status, claim_date);
Query OK, 0 rows affected (0.18 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SHOW INDEX FROM CLAIM;
+-------+------------+--------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table | Non_unique | Key_name                 | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-------+------------+--------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| claim |          0 | PRIMARY                  |            1 | claim_id    | A         |           6 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| claim |          0 | incident_id              |            1 | incident_id | A         |           6 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| claim |          1 | policy_id                |            1 | policy_id   | A         |           6 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| claim |          1 | idx_claim_status_date    |            1 | claim_status| A         |           4 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| claim |          1 | idx_claim_status_date    |            2 | claim_date  | A         |           6 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+-------+------------+--------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
5 rows in set (0.03 sec)

mysql> EXPLAIN
    -> SELECT *
    -> FROM CLAIM
    -> WHERE claim_status = 'Pending'
    -> AND claim_date >= '2026-01-01';
+----+-------------+-------+------------+-------+--------------------------+--------------------------+---------+------+------+----------+-----------------------+
| id | select_type | table | partitions | type  | possible_keys            | key                      | key_len | ref  | rows | filtered | Extra                 |
+----+-------------+-------+------------+-------+--------------------------+--------------------------+---------+------+------+----------+-----------------------+
|  1 | SIMPLE      | CLAIM | NULL       | range | idx_claim_status_date    | idx_claim_status_date    | 125     | NULL |    1 |   100.00 | Using index condition |
+----+-------------+-------+------------+-------+--------------------------+--------------------------+---------+------+------+----------+-----------------------+
1 row in set, 1 warning (0.02 sec)
```

composite index named idx_claim_status_date was created on the CLAIM table to support efficient filtering of insurance claims based on their status and claim date.

3. Transactions with COMMIT and ROLLBACK

inserting a temporary customer to demonstrate COMMIT

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO CUSTOMER
    -> (customer_id, name, date_of_birth, gender, phone, email, address)
    -> VALUES
    -> (9998, 'Transaction Test Customer', '2000-01-01',
    ->  'Other', '9999999998',
    ->  'transaction.test@example.com', 'Bengaluru');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT *
    -> FROM CUSTOMER
    -> WHERE customer_id = 9998;
+-------------+---------------------------+---------------+--------+------------+-----------------------------+-----------+
| customer_id | name                      | date_of_birth | gender | phone      | email                       | address   |
+-------------+---------------------------+---------------+--------+------------+-----------------------------+-----------+
|        9998 | Transaction Test Customer | 2000-01-01    | Other  | 9999999998 | transaction.test@example.com | Bengaluru |
+-------------+---------------------------+---------------+--------+------------+-----------------------------+-----------+
1 row in set (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT *
    -> FROM CUSTOMER
    -> WHERE customer_id = 9998;
+-------------+---------------------------+---------------+--------+------------+-----------------------------+-----------+
| customer_id | name                      | date_of_birth | gender | phone      | email                       | address   |
+-------------+---------------------------+---------------+--------+------------+-----------------------------+-----------+
|        9998 | Transaction Test Customer | 2000-01-01    | Other  | 9999999998 | transaction.test@example.com | Bengaluru |
+-------------+---------------------------+---------------+--------+------------+-----------------------------+-----------+
1 row in set (0.00 sec)

mysql>
```

starting transaction ,inserting a temporary customer ,checking records, and then permanently saving the records by committing. then verifying if the commit was successful
hence the COMMIT command was demonstrated by permanently saving a newly inserted customer record within a transaction.

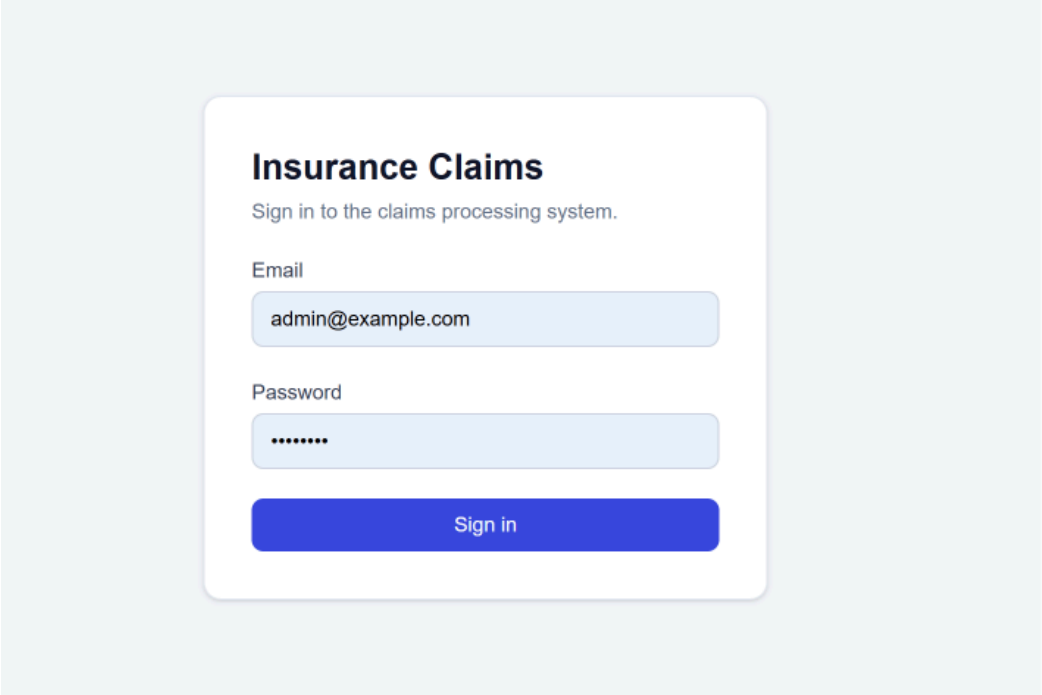
then we start another transaction and modify that same temporary customer's address , check the temp change , and undo the change and then verify again if the address is reverted back

SUMMARY OF ALL THE MISCELLANEOUS QUERIES

The miscellaneous queries demonstrate additional database operations performed on the Insurance Claims Processing System. These include granting database access to a teammate and establishing remote connectivity using an IP address and port number. Various JOIN operations, GROUP BY, HAVING and aggregate functions were used to retrieve policy details, identify customers with multiple policies, calculate claim averages and compare claimed and assessed amounts. Subqueries were used to identify claims above the average amount, while other queries analysed claim statuses, settlement percentages and pending settlements. A stored procedure and function were used to retrieve claim details and calculate settlement percentages. Additionally, a view was created for consolidated claim information, a composite index was implemented to support claim searches, and COMMIT and ROLLBACK operations demonstrated transaction management.

10. FRONT END DEVELOPMENT

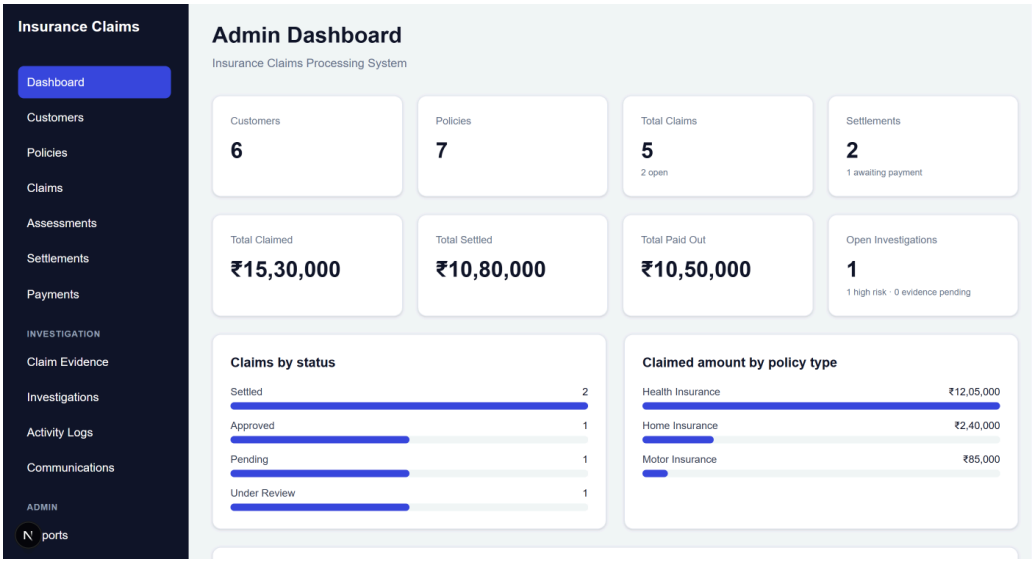
Login page:

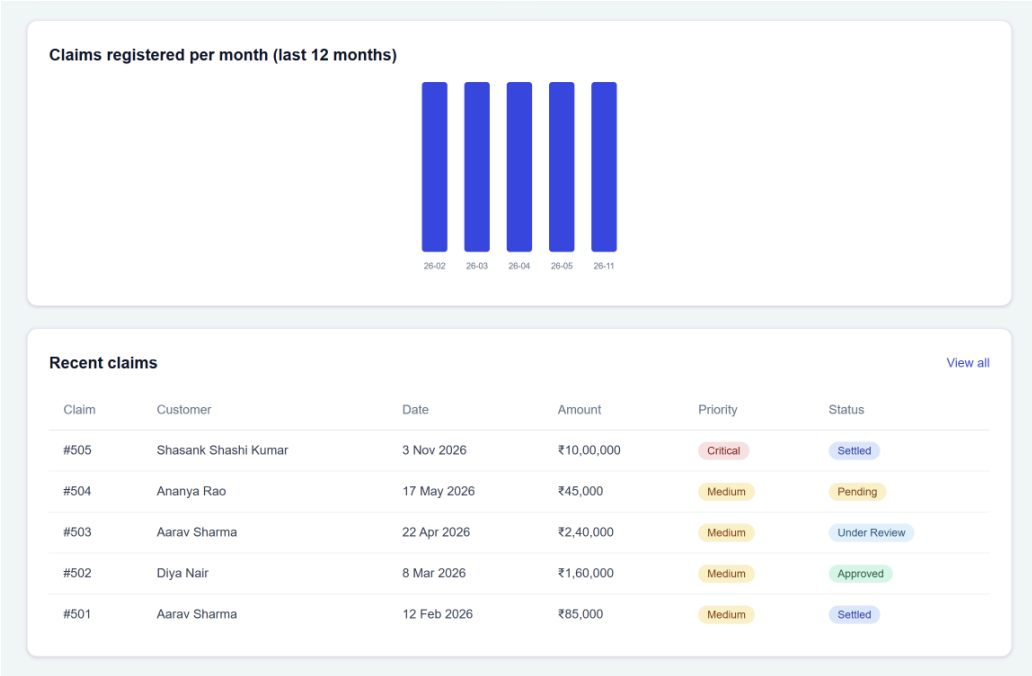


Admins and users can login with their registered mail address.
Note: this page is for admins/ users created from backend with email address and password assigned
Admins can create new users who have signed up for an insurance policy.

Dashboard:

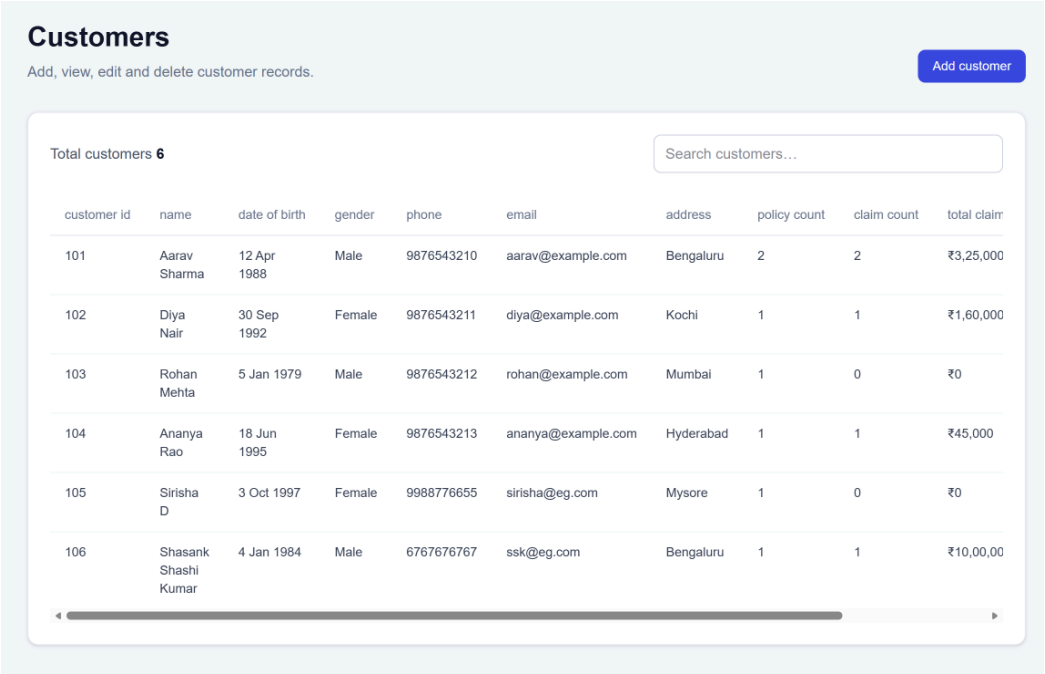
This view is for admin to view stats such as number of customers, policies, claims, settlement amounts.
Recent claim information can also be viewed along with analytics which would be useful in the business.





Customers:

Admin can add, view and delete customers



GitHubLink: https://github.com/shenoyshre22/Insurance-Claims-database-

REFERENCES/BIBLIOGRAPHY

used the following tools in the making of the project hence their documentation is attached below

1. MySQL Documentation. https://dev.mysql.com/doc/
2. Next.js Documentation. https://nextjs.org/docs
3. React Documentation. https://react.dev/
4. Tailwind CSS Documentation. https://tailwindcss.com/docs
5. GitHub Copilot Documentation. https://docs.github.com/en/copilot
6. OpenAI Codex Documentation. https://developers.openai.com/codex/
7. PES University, DBMS Semester 5 Mini Project Guidelines and Lab Materials.

APPENDIX A: DEFINITIONS, ACRONYMS AND ABBREVIATIONS

Term	Definition
DBMS	Database Management System
SQL	Structured Query Language
DDL	Data Definition Language
DML	Data Manipulation Language
DCL	Data Control Language
TCL	Transaction Control Language
ER	Entity Relationship
PK	Primary Key
FK	Foreign Key
CRUD	Create, Read, Update, Delete
RDBMS	Relational Database Management System
NoSQL	Non-relational Database
API	Application Programming Interface
UI	User Interface
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript

TS	TypeScript
SQL JOIN	Operation used to combine rows from related tables
Subquery	A query nested inside another SQL query
Correlated Query	A subquery that references columns from its outer query
Stored Procedure	A reusable set of SQL statements stored in the database
Stored Function	A stored database routine that returns a value
Trigger	A database operation executed automatically in response to a specified event
View	A virtual table based on an SQL query
Index	A database structure used to improve data retrieval
Transaction	A group of database operations treated as a single unit
COMMIT	Permanently saves transaction changes
ROLLBACK	Reverses uncommitted transaction changes
MySQL	Relational Database Management System
Next.js	React-based web application framework
React	JavaScript library for building user interfaces
Tailwind CSS	Utility-first CSS framework